

```
1 var _diagMsgs = [];
2 var _diagStart = Date.now();
3 var _diagModalShowing = false;
4 var _diagModalShown = false;
5 function _diag(msg) {
6   var line = '[' + (Date.now() - _diagStart) + 'ms]' + msg;
7   _diagMsgs.push(line);
8   try { console.log(line); } catch (_) {}
9 }
10 function _showDiag() {
11   try {
12     if (_diagModalShowing || _diagModalShown) {
13       return;
14     }
15     if (typeof wx !== 'undefined' && wx.showModal) {
16       _diagModalShowing = true;
17       _diagModalShown = true;
18       wx.showModal({
19         title: '启动诊断',
20         content: _diagMsgs.join('\n'),
21         showCancel: false,
22         fail: function(err) {
23           try { console.warn('[diag] showModal failed', err); } catch (_) {}
24         },
25         complete: function() {
26           _diagModalShowing = false;
27         }
28       });
29     }
30   } catch (_) {}
31 }
32 _diag('game.js start');
33 try {
34   if (typeof wx !== 'undefined' && typeof wx.createCanvas === 'function' && typeof GameGlobal !== ...
35   // 借鉴 xiao_chu 的好友榜方案：第一个 createCanvas 作为真正上屏 Canvas，并锁定 2D context。
36   // 微信开放数据域 sharedCanvas 只能 drawImage 到上屏 Canvas，不能进 WebGL/普通离屏 Canvas。
37   // pixi-adapter 会在后面再创建一个 canvas 给 Pixi/WebGL 使用，Game.ts 每帧把 Pixi 离屏画面
38   // 与 sharedCanvas 合成到这个 2D 上屏 Canvas。
39   var _mainCanvas = wx.createCanvas();
40   var _mainCtx = _mainCanvas.getContext('2d');
41   var _screenInfo = wx.getSystemInfoSync ? wx.getSystemInfoSync() : null;
42   var _dpr = (_screenInfo && _screenInfo.pixelRatio) || 2;
43   var _screenW = (_screenInfo && (_screenInfo.windowWidth || _screenInfo.screenWidth)) || 375;
44   var _screenH = (_screenInfo && (_screenInfo.windowHeight || _screenInfo.screenHeight)) || 667;
45   _mainCanvas.width = _screenW * _dpr;
46   _mainCanvas.height = _screenH * _dpr;
47   if (_mainCtx) {
48     _mainCtx.fillStyle = '#f7e4c4';
49     _mainCtx.fillRect(0, 0, _mainCanvas.width, _mainCanvas.height);
50   }
```

```
51   GameGlobal.__mainCanvas = _mainCanvas;
52   _diag('main 2d canvas ready:' + _mainCanvas.width + 'x' + _mainCanvas.height);
53 }
54 } catch (e) {
55   _diag('main 2d canvas failed:' + e);
56 }
57 try {
58   if (typeof wx !== 'undefined' && wx.getSystemInfoSync) {
59     var _info = wx.getSystemInfoSync();
60     _diag('platform:' + _info.platform + ' system:' + _info.system);
61     _diag('brand:' + _info.brand + ' model:' + _info.model);
62     _diag('SDK:' + _info.SDKVersion);
63   }
64 } catch (e) {
65   _diag('systemInfo failed:' + e);
66 }
67 try {
68   if (typeof GameGlobal !== 'undefined') {
69     GameGlobal.onError = function(msg) {
70       _diag('onError:' + msg);
71       _showDiag();
72     };
73     GameGlobal.onUnhandledRejection = function(ev) {
74       _diag('unhandledRejection:' + (ev && ev.reason || ev));
75       _showDiag();
76     };
77   }
78 } catch (_) {}
79 try {
80   require('./minigame/pixi-adapter/index');
81   _diag('pixi-adapter ok');
82 } catch (e) {
83   _diag('pixi-adapter failed:' + e);
84   _showDiag();
85 }
86 // 部分安卓/鸿蒙微信小游戏 JS 引擎没有 Intl, PixiJS 文本测量里直接访问 Intl 会导致启动阶段 ReferenceError。
87 if (typeof Intl === 'undefined') {
88   _diag('Intl missing, install stub');
89   try {
90     if (typeof GameGlobal !== 'undefined') GameGlobal.Intl = {};
91     if (typeof globalThis !== 'undefined') globalThis.Intl = {};
92   } catch (_) {}
93 }
94 try {
95   require('./minigame/game-bundle.js');
96   _diag('game-bundle ok');
97 } catch (e) {
98   _diag('game-bundle failed:' + e);
99   _showDiag();
100 }
```

```
101 try {
102   setTimeout(function() {
103     if (typeof GameGlobal !== 'undefined' && !GameGlobal.__hotPotRendered) {
104       _diag('5s timeout: game not rendered');
105       _showDiag();
106     }
107   }, 5000);
108 } catch (_) {}
109 import { Analytics, EVENT_NAMES, type DeviceInfo, type PlatformName } from '@gp/analytics-sdk';
110 import { BACKEND_BASE_URL } from '@config/CloudConfig';
111 import { Platform } from '@core/PlatformService';
112 export { EVENT_NAMES };
113 export const analytics = Analytics;
114 const APP_VERSION = '1.0.0';
115 const GAME_KEY = 'hotpot';
116 /**
117  * 复用 hot-pot 现有 hotpot-api 同款 HTTP 访问服务网关:
118  * <env-id>.service.tcloudbase.com/<云函数名>/<path>
119  * 这是 CloudBase 「HTTP 访问服务」网关, 跟「云函数 HTTP 触发器」是两套不同 URL 格式,
120  * 之前写的 *.app.tcloudbase.com/... 是后者的格式, 会被网关 404 INVALID_PATH 拒绝。
121  */
122 const ENDPOINT = `${BACKEND_BASE_URL}/analytics-ingest/track`;
123 let inited = false;
124 /** hot-pot 接入入口: 在 main.ts 启动尽早调用一次。把 PlatformService 作为 Adapter 注入给 SDK。 */
125 export function initAnalytics(opts?: { endpoint?: string; userId?: string; debug?: boolean }): void {
126   if (inited) return;
127   const platformName = mapPlatform();
128   const deviceInfo = buildDeviceInfo();
129   Analytics.init({
130     endpoint: opts?.endpoint || ENDPOINT,
131     gameKey: GAME_KEY,
132     appVersion: APP_VERSION,
133     platform: platformName,
134     deviceInfo,
135     initialUserId: opts?.userId,
136     transport: { request: Platform.request.bind(Platform) },
137     storage: {
138       get: Platform.getStorageSync.bind(Platform),
139       set: Platform.setStorageSync.bind(Platform),
140       remove: Platform.removeStorageSync.bind(Platform),
141     },
142     lifecycle: { onHide: Platform.onHide.bind(Platform) },
143     debug: opts?.debug,
144   });
145   inited = true;
146 }
147 /** 业务登录拿到 openid 后调用, 让后续事件都带上 user_id */
148 export function setAnalyticsUserId(userId: string): void {
149   if (!inited) return;
150   Analytics.setUserId(userId || '');
151 }
```

```
151 }
152 function mapPlatform(): PlatformName {
153   if (Platform.isWechat) return 'wechat';
154   if (Platform.isDouyin) return 'douyin';
155   if (Platform.isMinigame) return 'unknown';
156   return 'h5';
157 }
158 function buildDeviceInfo(): DeviceInfo {
159   const sys = Platform.getSystemInfoSync() || {};
160   return {
161     brand: String(sys.brand || sys.deviceBrand || ''),
162     model: String(sys.model || sys.deviceModel || ''),
163     system: String(sys.system || ''),
164     sdkVersion: String(sys.SDKVersion || sys.sdkVersion || ''),
165     screenWidth: Number(sys.screenWidth) || 0,
166     screenHeight: Number(sys.screenHeight) || 0,
167     network: 'unknown',
168   };
169 }
170 /** CloudBase HTTP backend configuration for the hotpot game. */
171 export const GAME_KEY = 'hotpot';
172 export const BACKEND_BASE_URL = 'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com';
173 export const BACKEND_PATH_PREFIX = `/${GAME_KEY}-api`;
174 export const CLOUD_ENV_ID = 'rosa-env-d7grf78r5dbd37323';
175 export const BACKEND_LOGIN_PATH = `${BACKEND_PATH_PREFIX}/login/`;
176 export const BACKEND_PULL_PATH = `${BACKEND_PATH_PREFIX}/save/pull`;
177 export const BACKEND_PUSH_PATH = `${BACKEND_PATH_PREFIX}/save/push`;
178 export const BACKEND_HEALTH_PATH = `${BACKEND_PATH_PREFIX}/health`;
179 export const BACKEND_RANK_SUBMIT_PATH = `${BACKEND_PATH_PREFIX}/rank/submit`;
180 export const BACKEND_RANK_LIST_PATH = `${BACKEND_PATH_PREFIX}/rank/list`;
181 export const BACKEND_RANK_MINE_PATH = `${BACKEND_PATH_PREFIX}/rank/mine`;
182 export const BACKEND_LEVEL_PASS_RATES_PATH = `${BACKEND_PATH_PREFIX}/level/pass-rates`;
183 export const BACKEND_GAME_CLUB_DECRYPT_PATH = `${BACKEND_PATH_PREFIX}/game-club/decrypt`;
184 export const LEVEL_PASS_RATE_COLLECTION = 'hotpot_public_level_pass_rates';
185 export const LEVEL_PASS_RATE_DOC_ID = 'bowl_30d_latest';
186 export const BACKEND_REQUEST_TIMEOUT_MS = 10000;
187 export const BACKEND_TOKEN_KEY = `${GAME_KEY}_token`;
188 export const BACKEND_ANON_ID_KEY = `${GAME_KEY}_anon_id`;
189 export const BOWL_PROGRESS_KEY = `${GAME_KEY}_bowl_progress`;
190 export const USER_SETTINGS_KEY = `${GAME_KEY}_settings`;
191 export const FRUIT_SLICE_PROGRESS_KEY = `${GAME_KEY}_fruit_slice_progress`;
192 export const WALLET_KEY = `${GAME_KEY}_wallet_v1`;
193 export const GACHA_STATE_KEY = `${GAME_KEY}_gacha_state_v1`;
194 export const TOOL_INVENTORY_KEY = `${GAME_KEY}_tool_inventory_v1`;
195 export const FRUIT_SLICE_REWARD_KEY = `${GAME_KEY}_fruit_slice_reward_v1`;
196 export const FRUIT_SLICE_TOOL_INVENTORY_KEY = `${GAME_KEY}_fruit_slice_tool_inventory_v1`;
197 export const GAME_CLUB_REWARD_KEY = `${GAME_KEY}_game_club_reward_v1`;
198 export const LEVEL_PASS_RATE_CACHE_KEY = `${GAME_KEY}_level_pass_rates_v1`;
199 /**
200  * 玩家自己授权拿到的微信昵称 + 头像 URL；仅用于排行榜显示。
```

```
201 * 不进入 CLOUD_SYNC_ALLOWLIST: 用户资料本机敏感, 不参与跨设备云端同步,
202 * 换设备时由玩家在新设备上重新点击授权即可。
203 */
204 export const USER_PROFILE_KEY = `${GAME_KEY}_user_profile`;
205 export const CLOUD_SYNC_SCHEMA_VERSION = 1;
206 export const CLOUD_SYNC_META_KEY = `${GAME_KEY}_cloud_meta`;
207 export const CLOUD_SYNC_ALLOWLIST = [
208   BOWL_PROGRESS_KEY,
209   USER_SETTINGS_KEY,
210   FRUIT_SLICE_PROGRESS_KEY,
211   WALLET_KEY,
212   GACHA_STATE_KEY,
213   TOOL_INVENTORY_KEY,
214   FRUIT_SLICE_REWARD_KEY,
215   FRUIT_SLICE_TOOL_INVENTORY_KEY,
216   GAME_CLUB_REWARD_KEY,
217 ] as const;
218 export const CLOUD_SYNC_EXCLUDE_KEYS = [
219   BACKEND_TOKEN_KEY,
220   BACKEND_ANON_ID_KEY,
221 ] as const;
222 export const CLOUD_SYNC_STARTUP_TIMEOUT_MS = 2500;
223 export const CLOUD_SYNC_DEBOUNCE_MS = 1500;
224 export const CLOUD_SYNC_BASE_DELAY_MS = 1500;
225 export const CLOUD_SYNC_MAX_BACKOFF_MS = 30000;
226 export const CLOUD_SYNC_MAX_FAIL_COUNT = 5;
227 export const CLOUD_SYNC_RETRY_INTERVAL_MS = 60000;
228 export const CLOUD_SYNC_LOG_THRESHOLD = 3;
229 export type CloudSyncKey = typeof CLOUD_SYNC_ALLOWLIST[number];
230 /**
231 * 玩法相关贴图放在微信分包, 降低主包体积 (主包仅保留首页等) 。
232 * 路径相对小游戏项目根目录 (与 game.json 中 subpackages.bowl_game 一致) 。
233 */
234 export const BOWL_IMAGES_ROOT = 'subpackages/bowl_game/assets/images';
235 export const BOWL_CORE_IMAGES_ROOT = 'subpackages/bowl_core/assets/images';
236 export const BOWL_THEME_IMAGES_ROOT = 'subpackages/bowl_themes/assets/images/themes';
237 export const BOWL_BADGE_IMAGES_ROOT = 'subpackages/bowl_badges/assets/images/badges';
238 export const CATALOG_IMAGES_ROOT = 'subpackages/catalog_assets/assets/images';
239 export const DAILY_LIMITED_IMAGES_ROOT = 'subpackages/daily_limited/assets/images/daily_limited';
240 export const DAILY_RECIPE_IMAGES_ROOT = 'subpackages/daily_recipes/assets/images/daily_limited/re...
241 export const FRUIT_SLICE_IMAGES_ROOT = 'subpackages/fruit_slice/assets/images/fruit_slice';
242 export const GACHA_IMAGES_ROOT = 'subpackages/gacha_assets/assets/images/gacha';
243 import { BOWL_BADGE_IMAGES_ROOT } from '@config/bowlAssets';
244 export interface BowlBadgeDef {
245   levelNumber: number;
246   title: string;
247   asset: string;
248   vessel: 'cup' | 'bowl';
249   drinkColor: number;
250   accentColor: number;
```

```
251   fruitColors: readonly number[];
252   garnishColor: number;
253 }
254 const badge = (
255   levelNumber: number,
256   title: string,
257   vessel: BowlBadgeDef['vessel'],
258   drinkColor: number,
259   accentColor: number,
260   fruitColors: readonly number[],
261   garnishColor: number,
262 ): BowlBadgeDef => ({
263   levelNumber,
264   title,
265   asset: `${BOWL_BADGE_IMAGES_ROOT}/${bowl_badge_${String(levelNumber).padStart(2, '0')}}.png`,
266   vessel,
267   drinkColor,
268   accentColor,
269   fruitColors,
270   garnishColor,
271 });
272 export const BOWL_BADGES: BowlBadgeDef[] = [
273   badge(1, '酸奶莓果杯', 'cup', 0xffff4e7, 0x7ed8ff, [0xff5a7a, 0x355fd1, 0xffc94d], 0x65c85a),
274   badge(2, '柠檬蓝莓冰茶', 'cup', 0xffe66d, 0x68d6ff, [0xffe34d, 0x3c73df, 0xff7aaa], 0x50bc6b),
275   badge(3, '热带芒芒刨冰', 'bowl', 0xffb43d, 0xffec88, [0xff9f2f, 0xff4c61, 0xffdd4c], 0x62c765),
276   badge(4, '星星西瓜水果捞', 'bowl', 0xff6b82, 0x93f0ff, [0xff3f63, 0x3ed56b, 0xffe95c], 0x48b65c),
277   badge(5, '莓柚气泡冰杯', 'cup', 0xff7aa8, 0xb8efff, [0xff4b7d, 0xbb3fe3, 0xffc34f], 0x71cf64),
278   badge(6, '坚果蜜桃雪顶杯', 'cup', 0xffc17f, 0xffff0b3, [0xffa46b, 0xd8a05a, 0xffe087], 0x78bd52),
279   badge(7, '椰香桂圆冰碗', 'bowl', 0xffead0, 0x8fe7ff, [0xffffff, 0xe7a858, 0xffd967], 0x7ac568),
280   badge(8, '多彩小料奶茶', 'cup', 0xd9a06a, 0x83ddff, [0xff7acd, 0xffd54a, 0x6c55d8], 0x58bd70),
281   badge(9, '脆脆莓莓冰沙', 'bowl', 0xf55e92, 0xbdf4ff, [0xff5280, 0xffd447, 0x8e5ee7], 0x75c85d),
282   badge(10, '双轨橙柚冰茶', 'cup', 0xff9d35, 0x68d8ff, [0xffd24a, 0xff6b42, 0x7bd85a], 0x5bc46d),
283   badge(11, '十二鲜果冰碗', 'bowl', 0xffefb4, 0x9befff, [0xff515f, 0x3c7ee8, 0xffdc54, 0x70d95d], 0x60b...
284   badge(12, '续章荔枝水果茶', 'cup', 0xffd9e8, 0x96e8ff, [0xff78a2, 0xffff2f2, 0xffc14f], 0x6ac66a),
285   badge(13, '东方梅子冰饮', 'cup', 0xb95fc2, 0xffd87b, [0x8a3f9e, 0xff6b86, 0xffe066], 0x64bd70),
286   badge(14, '混搭雪梨刨冰', 'bowl', 0xf7f5dc, 0x89ddff, [0xffd552, 0xff7a4d, 0x7edc70], 0x5ec06c),
287   badge(15, '蜜瓜蜜桃冰杯', 'cup', 0xbef17a, 0xffb7d0, [0xff9a60, 0xc8f36a, 0xffe45a], 0x5abf65),
288   badge(16, '满贯浆果雪山', 'bowl', 0xd46be8, 0xb5efff, [0xff507b, 0x4f6ce8, 0xffd557], 0x73ca62),
289   badge(17, '满贯椰椰冰茶', 'cup', 0xffff4d6, 0x7ee0ff, [0xffffff, 0xffcc58, 0x7bd961], 0x55bd67),
290   badge(18, '杂烩水果捞碗', 'bowl', 0xffc25f, 0x9defff, [0xff5b61, 0x7f56df, 0xffe156, 0x5fce65], 0x58b...
291   badge(19, '薄冰青柠杯', 'cup', 0xcff377, 0xa9f3ff, [0xc6f04e, 0xffffff, 0x68d8ff], 0x51b96a),
292   badge(20, '四合莲藕冰碗', 'bowl', 0xf2dfc9, 0x88ddff, [0xf2c28a, 0xffe06a, 0xb87bdc], 0x6bc15f),
293   badge(21, '果阵红莓冰沙', 'cup', 0xff5d73, 0xffd1e0, [0xff3f5d, 0x405bdc, 0xffd54d], 0x68c767),
294   badge(22, '果阵青柠雪碗', 'bowl', 0xaee95e, 0x8ee7ff, [0xb9ef4e, 0xffdd4a, 0x68d9ff], 0x56bf68),
295   badge(23, '滋补银耳甜品', 'bowl', 0xffefd5, 0xf6c17d, [0xffffff, 0xc06ad9, 0xffd760], 0x7ac160),
296   badge(24, '小料珍珠奶茶', 'cup', 0xcf9863, 0x83dbff, [0x5a3428, 0xff7fc8, 0xffdc56], 0x62bd66),
297   badge(25, '重口巧克力冰杯', 'cup', 0x8b5a38, 0xffc968, [0x5b3524, 0xffff2e0, 0xff6a84], 0x68bd59),
298   badge(26, '果下芒果椰雪', 'bowl', 0xffc53f, 0xffef91, [0xffa438, 0xffffff, 0xffdd4a], 0x65c45d),
299   badge(27, '滋补红枣冰饮', 'cup', 0xb84f45, 0xffd18a, [0xa64038, 0xffda65, 0xf4eee0], 0x6cc062),
300   badge(28, '小料仙草冰碗', 'bowl', 0x493d45, 0x8bdcff, [0x302934, 0xffc84a, 0xff78c7], 0x65bd67),
```

```

301 badge(29, '甘星爆珠水果茶', 'cup', 0xff8b47, 0x92e9ff, [0xff3f6d, 0xffdb4a, 0x5f70e8, 0x61d35b], 0x5ac...
302 badge(30, '终章龙果冰冠', 'bowl', 0xff56aa, 0xffe06a, [0xff2f91, 0xffffff, 0xffd84e, 0x63d966], 0x59c...
303 badge(31, '紫冠雪梨杯', 'cup', 0xf4e8ff, 0xffe58a, [0x6a1b9a, 0xf6e7a8, 0xffffff], 0x78c968),
304 badge(32, '牛油果奶昔杯', 'cup', 0xc8f0a8, 0xfffd4d, [0x7fba3f, 0xf5efcf, 0x5c7a2f], 0x66bd5f),
305 badge(33, '红宝石榴冰', 'bowl', 0xd9324c, 0xffd78a, [0xc91f3c, 0xff6f86, 0xffff1d6], 0x5fc36b),
306 badge(34, '弯月山楂冰茶', 'cup', 0xffd77a, 0xff8b7a, [0xbce874, 0xd73338, 0xffff0a8], 0x63bf63),
307 badge(35, '莲雾粉雾杯', 'cup', 0xffb8c8, 0xa8efff, [0xff5f86, 0xffd6e0, 0xffffff], 0x72c76a),
308 badge(36, '油柑回甘冰饮', 'cup', 0xbde86b, 0xffdf72, [0x90c84a, 0xdff59a, 0xffff0b8], 0x5ebd68),
309 badge(37, '芭乐餐车奶昔', 'cup', 0xd9f0a8, 0xffcfa8, [0x9fd66a, 0xffb6b6, 0xffffff], 0x62bf68),
310 badge(38, '木瓜暖奶碗', 'bowl', 0xffb15a, 0xffefd1, [0xff8f2e, 0xffcf67, 0xffffff], 0x67c35d),
311 badge(39, '无花果蜜宴碗', 'bowl', 0xb85a93, 0xffc76f, [0xd3a8c, 0xd96f9f, 0xffe08a], 0x6bc064),
312 badge(40, '杏光沙滩终宴', 'bowl', 0xffb55a, 0xffe077, [0xff9c3d, 0xffd06a, 0xffff2c0, 0x63d966], 0x58c...
313 ];
314 export function getBowlBadgeDef(levelNumber: number): BowlBadgeDef {
315     const clamped = Math.max(1, Math.min(levelNumber, BOWL_BADGES.length));
316     return BOWL_BADGES[clamped - 1];
317 }
318 import type { FruitId } from '@config/fruits';
319 import type { BowlRimKey, BowlSoupKey } from '@config/bowlSkins';
320 import type { BowlThemeKey } from '@config/bowlThemes';
321 export interface BowlLevelDef {
322     /** 关卡序号 (从 1 起, 与 UI「第 N 关」一致) */
323     levelNumber: number;
324     displayName: string;
325     /** 本关出现的可下单食材种类 */
326     fruitIds: FruitId[];
327     /** 每种可下单食材在碗内出现的个数 */
328     copiesPerFruit: number;
329     /** 每个订单需要的个数 */
330     orderTarget: number;
331     /** 底部暂存槽位数 */
332     bufferSize: number;
333     /** 冰块障碍件数: 不进订单, 但会占碗与暂存压力 */
334     iceCount?: number;
335     /**
336      * 冻果件数: 从本关订单库存中挑 N 颗盖上冰块。
337      * 点击会强制进暂存槽并开始解冻; 解冻后按普通水果交付, 仍计入订单总数。
338      */
339     frozenCount?: number;
340     /** 开局浮在上层、可点击的物品上限; 其余先藏在下层, 随订单推进浮出 */
341     initialVisibleCount?: number;
342     /** 每完成一盘订单后, 从下层释放的物品数量 */
343     revealPerOrderComplete?: number;
344     /** 开局并行订单路数, 默认 2; 新手加速关可直接开到 4 路 */
345     plateLanesInitial?: 2 | 3 | 4;
346     allowAddDish: boolean;
347     allowRemove: boolean;
348     allowShuffle: boolean;
349     soupKey?: BowlSoupKey;
350     bowlKey?: BowlRimKey;

```

```
351  /** 玩法页主题；未配置时按关卡区间自动切换 */
352  themeKey?: BowlThemeKey;
353  }
354  const allTools = { allowAddDish: true, allowRemove: true, allowShuffle: true } as const;
355  /**
356  * 每关新解锁的食材（累积式：本关订单池 = 前 N 组并集）。
357  * 节奏：L1=4、L2 解锁 4 但订单池压到 7、L3 解锁 4 但订单池压到 11；
358  *   L4-15 保持原解锁节奏不动，L16-30 补齐全部小料/滋补/甜品食材，
359  *   L31-40 开启鲜果续章。
360  * 关卡通关弹窗的「下一关解锁」会显示对应组中的新水果。
361  */
362  const UNLOCK_GROUPS = [
363    ['blueberry', 'lemon', 'orange', 'strawberry'],
364    ['apple', 'banana', 'grape', 'kiwi'],
365    ['cucumber', 'peach', 'pineapple', 'watermelon'],
366    ['mango', 'cherry', 'cherry_tomato'],
367    ['grape_green', 'lime', 'mandarin'],
368    ['cantaloupe', 'honeydew', 'young_coconut'],
369    ['lychee', 'longan'],
370    ['dried_longan', 'bayberry'],
371    ['blackberry', 'cranberry'],
372    ['raspberry', 'mulberry'],
373    ['passionfruit', 'grapefruit'],
374    ['kumquat', 'starfruit'],
375    ['plum', 'nectarine'],
376    ['persimmon', 'almond_slice'],
377    ['peanut', 'walnut_piece'],
378    ['chestnut', 'oat_flake'],
379    ['red_date', 'sour_plum', 'mint'],
380    ['blackcurrant', 'gooseberry', 'osmanthus'],
381    ['basil_seed', 'crystal_jelly'],
382    ['lotus_root', 'water_chestnut', 'radish_heart'],
383    ['black_rice', 'red_bean'],
384    ['foxnut', 'lotus_seed'],
385    ['lily_bulb', 'snow_fungus', 'peach_gum'],
386    ['boba_pearl', 'pudding_cube', 'mini_mochi'],
387    ['chocolate_chip', 'cookie_crumb', 'marshmallow'],
388    ['pumpkin_cube', 'sweet_potato', 'durian'],
389    ['coconut_jelly', 'sago'],
390    ['grass_jelly', 'taro_ball', 'taro_dice'],
391    ['pop_boba'],
392    ['dragonfruit'],
393    ['mangosteen', 'pear'],
394    ['avocado'],
395    ['pomegranate'],
396    ['yangjiaomi_melon', 'hawthorn'],
397    ['wax_apple'],
398    ['emblic'],
399    ['guava'],
400    ['papaya'],
```



```
401  ['fig'],
402  ['apricot'],
403 ] as const satisfies readonly (readonly FruitId[]);
404 /** 图鉴只展示关卡体系内真正可获得的食材，避免通关后仍出现永远无法解锁的占位。 */
405 export const BOWL_UNLOCKABLE_FRUIT_IDS = Array.from(new Set<FruitId>(UNLOCK_GROUPS.flat()));
406 /**
407  * 后 15 关不再把最早期全部基础水果无限累积进订单池，而是保留一批辨识度高的主食材，
408  * 再叠加最近解锁的小料/滋补/甜品，保证每关新鲜感和图鉴解锁一致。
409  */
410 const POST_15_BASE_POOL = [
411   'apple',
412   'banana',
413   'grape',
414   'kiwi',
415   'peach',
416   'pineapple',
417   'watermelon',
418   'mango',
419   'cherry',
420   'cherry_tomato',
421   'grape_green',
422   'lime',
423   'mandarin',
424   'cantaloupe',
425   'honeydew',
426   'young_coconut',
427   'lychee',
428   'longan',
429   'bayberry',
430   'blackberry',
431   'cranberry',
432   'raspberry',
433   'mulberry',
434   'passionfruit',
435   'grapefruit',
436   'plum',
437   'nectarine',
438   'persimmon',
439   'almond_slice',
440   'peanut',
441   'walnut_piece',
442 ] as const satisfies readonly FruitId[];
443 const POST_15_RECENT_GROUP_WINDOW = 10;
444 const POST_15_FIRST_LEVEL = 16;
445 const POST_15_START_ORDER_COUNT = 54;
446 const POST_15_ORDER_INCREMENT = 1;
447 const POST_15_COPIES_PER_FRUIT = 6;
448 const FINAL_CHAPTER_FIRST_LEVEL = 31;
449 const FINAL_CHAPTER_START_ORDER_COUNT = 72;
450 const FINAL_CHAPTER_END_ORDER_COUNT = 100;
```

```

451 function uniqueFruitIds(ids: readonly FruitId[]): FruitId[] {
452   return Array.from(new Set<FruitId>(ids));
453 }
454 function post15TargetFoodCount(levelNumber: number): number {
455   const orderCount = post15TargetOrderCount(levelNumber);
456   return Math.ceil((orderCount * 3) / POST_15_COPIES_PER_FRUIT);
457 }
458 function post15TargetOrderCount(levelNumber: number): number {
459   if (levelNumber >= FINAL_CHAPTER_FIRST_LEVEL) {
460     const progress = Math.max(0, Math.min(1, (levelNumber - FINAL_CHAPTER_FIRST_LEVEL) / 9));
461     return Math.round(
462       FINAL_CHAPTER_START_ORDER_COUNT +
463       (FINAL_CHAPTER_END_ORDER_COUNT - FINAL_CHAPTER_START_ORDER_COUNT) * progress,
464     );
465   }
466   return POST_15_START_ORDER_COUNT + (levelNumber - POST_15_FIRST_LEVEL) * POST_15_ORDER_INCREMENT;
467 }
468 function targetFruitCountForEarlyLevel(levelNumber: number): number {
469   const targets = [4, 7, 10, 12, 14, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25];
470   return targets[Math.max(0, Math.min(levelNumber - 1, targets.length - 1))];
471 }
472 function trimOldFruitsForEarlyLevel(levelIndex: number, fruits: FruitId[]): FruitId[] {
473   const target = targetFruitCountForEarlyLevel(levelIndex + 1);
474   if (fruits.length <= target) {
475     return fruits;
476   }
477   /**
478    * 降低每关水果种类时，保护本关新解锁食材；L3 起额外保护上一关食材。
479    * 其余名额从较新的旧食材往回补，优先剔除更早的旧水果。
480    */
481   const protectedStart = levelIndex <= 1 ? levelIndex : levelIndex - 1;
482   const protectedIds = new Set<FruitId>(uniqueFruitIds(UNLOCK_GROUPS.slice(protectedStart, levelIndex)));
483   const keep = new Set<FruitId>(protectedIds);
484   for (const id of fruits.filter((fruitId) => !protectedIds.has(fruitId)).reverse()) {
485     if (keep.size >= target) {
486       break;
487     }
488     keep.add(id);
489   }
490   return fruits.filter((id) => keep.has(id));
491 }
492 /**
493  * 累积式：第 N 关订单池 = UNLOCK_GROUPS[0..N-1] 全部并集（去重）。
494  * L16 起改为「后期基础池 + 最近新解锁食材」，避免早期食材过度重复。
495  */
496 function levelFruits(levelNumber: number): FruitId[] {
497   const idx = Math.max(0, Math.min(levelNumber - 1, UNLOCK_GROUPS.length - 1));
498   if (idx < 15) {
499     return trimOldFruitsForEarlyLevel(
500     idx,

```

```

501   uniqueFruitIds(UNLOCK_GROUPS.slice(0, idx + 1).flat()),
502   );
503 }
504 const recentStart = Math.max(15, idx - POST_15_RECENT_GROUP_WINDOW + 1);
505 const targetFoodCount = Math.max(
506   1,
507   levelNumber >= FINAL_CHAPTER_FIRST_LEVEL
508   ? post15TargetFoodCount(levelNumber)
509   : post15TargetFoodCount(levelNumber) - 1,
510 );
511 return uniqueFruitIds([
512   ...UNLOCK_GROUPS.slice(recentStart, idx + 1).flat(),
513   ...POST_15_BASE_POOL,
514   ...UNLOCK_GROUPS.slice(0, recentStart).flat(),
515 ]).slice(0, targetFoodCount);
516 }
517 /**
518  * 40 关数值 (v8, 平缓学习曲线 + 碗内"满当当"视觉) :
519  * `orderTarget` 全程 = 3 (三消核心玩法不动) ;
520  * `copiesPerFruit` 必须全程为 3 的倍数, 避免生成无法凑满 x3 的尾数水果。
521  * `plateLanesInitial` 默认 = 2 -- 第 3/4 路订单盘上的「解锁」按钮点了才看广告解锁
522  * L2 例外: 首个扩展教学关直接开 4 路, 让玩家体验多订单更快过关;
523  * (`unlockNextOrderPlateReward`) ; 失败复活也会顺带解锁一路。这是核心广告变现位。
524  * 底部「加菜碟」工具是另一条广告点: 每次 +1 个 `bufferSize` (最多 7) ,
525  * 缓解 buffer 满压力但不解锁订单路。
526  * 难度递增完全靠: 水果种类 ^、copiesPerFruit、冰 / 冻果 ^、bufferSize v、初始可见 ^。
527  *
528  * A 段 教学 (L1-2)      - 零障碍上手; L2 起 7 种水果让选择压力出现
529  * B 段 习惯养成 (L3-7)  - L3 起只增加水果种类与少量冰块, 不再把订单量翻倍
530  * C 段 中阶 (L8-15)    - 逐步加入冻果, 障碍缓慢爬升, 始终保留 5 个暂存盘
531  * D 段 长线 (L16-30)   - 订单量小幅增长, 靠新食材与背景变化提供新鲜感
532  * E 段 续章 (L31-40)   - 新鲜果回归, 叠加新碗汤奖励, 不再追加复杂机制
533  *
534  * 总订单单位 (`ordersRemaining`): L1=4 -> L3≈20 -> L10≈40 -> L15≈50 -> L16≈54 -> L30≈68 -> L40≈100
535  * `bufferSize`: 全程 5 格保留安全感; 加菜碟工具仍可扩到 7 格。
536  * `initialVisibleCount`: 跟随订单量慢慢提升; 通过水果放大和更多上层水果来保持画面丰富,
537  * 不再用过量可点击水果制造早期压迫感。
538  * `allTools` 全程开。
539  */
540 export const BOWL_LEVELS: BowlLevelDef[] = [
541   {
542     levelNumber: 1,
543     displayName: '第1关 酸奶初醒',
544     fruitIds: levelFruits(1),
545     copiesPerFruit: 3,
546     orderTarget: 3,
547     bufferSize: 5,
548     initialVisibleCount: 14,
549     revealPerOrderComplete: 4,
550     plateLanesInitial: 2,

```

```
551     ...allTools,
552   },
553   {
554     levelNumber: 2,
555     displayName: '第2关 鲜果开张',
556     fruitIds: levelFruits(2),
557     copiesPerFruit: 6,
558     orderTarget: 3,
559     bufferSize: 5,
560     initialVisibleCount: 32,
561     revealPerOrderComplete: 5,
562     plateLanesInitial: 4,
563     ...allTools,
564   },
565   {
566     levelNumber: 3,
567     displayName: '第3关 热带开席',
568     fruitIds: levelFruits(3),
569     copiesPerFruit: 6,
570     orderTarget: 3,
571     bufferSize: 5,
572     iceCount: 2,
573     initialVisibleCount: 54,
574     revealPerOrderComplete: 5,
575     plateLanesInitial: 2,
576     ...allTools,
577   },
578   {
579     levelNumber: 4,
580     displayName: '第4关 星果长廊',
581     fruitIds: levelFruits(4),
582     copiesPerFruit: 6,
583     orderTarget: 3,
584     bufferSize: 5,
585     iceCount: 3,
586     initialVisibleCount: 60,
587     revealPerOrderComplete: 5,
588     plateLanesInitial: 2,
589     ...allTools,
590   },
591   {
592     levelNumber: 5,
593     displayName: '第5关 薄冰试饮',
594     fruitIds: levelFruits(5),
595     copiesPerFruit: 6,
596     orderTarget: 3,
597     bufferSize: 5,
598     iceCount: 4,
599     frozenCount: 1,
600     initialVisibleCount: 66,
```

```
601   revealPerOrderComplete: 5,
602   plateLanesInitial: 2,
603   ...allTools,
604 },
605 {
606   levelNumber: 6,
607   displayName: '第6关 坚果蜜语',
608   fruitIds: levelFruits(6),
609   copiesPerFruit: 6,
610   orderTarget: 3,
611   bufferSize: 5,
612   iceCount: 4,
613   frozenCount: 1,
614   initialVisibleCount: 72,
615   revealPerOrderComplete: 5,
616   plateLanesInitial: 2,
617   ...allTools,
618 },
619 {
620   levelNumber: 7,
621   displayName: '第7关 暖盅小宴',
622   fruitIds: levelFruits(7),
623   copiesPerFruit: 6,
624   orderTarget: 3,
625   bufferSize: 5,
626   iceCount: 5,
627   frozenCount: 1,
628   initialVisibleCount: 78,
629   revealPerOrderComplete: 6,
630   plateLanesInitial: 2,
631   ...allTools,
632 },
633 {
634   levelNumber: 8,
635   displayName: '第8关 满席小酌',
636   fruitIds: levelFruits(8),
637   copiesPerFruit: 6,
638   orderTarget: 3,
639   bufferSize: 5,
640   iceCount: 5,
641   frozenCount: 2,
642   initialVisibleCount: 84,
643   revealPerOrderComplete: 6,
644   plateLanesInitial: 2,
645   ...allTools,
646 },
647 {
648   levelNumber: 9,
649   displayName: '第9关 甜脆交锋',
650   fruitIds: levelFruits(9),
```

```
651   copiesPerFruit: 6,
652   orderTarget: 3,
653   bufferSize: 5,
654   iceCount: 6,
655   frozenCount: 2,
656   initialVisibleCount: 90,
657   revealPerOrderComplete: 6,
658   plateLanesInitial: 2,
659   ...allTools,
660 },
661 {
662   levelNumber: 10,
663   displayName: '第10关 深汤追单',
664   fruitIds: levelFruits(10),
665   copiesPerFruit: 6,
666   orderTarget: 3,
667   bufferSize: 5,
668   iceCount: 6,
669   frozenCount: 2,
670   initialVisibleCount: 96,
671   revealPerOrderComplete: 6,
672   plateLanesInitial: 2,
673   ...allTools,
674 },
675 {
676   levelNumber: 11,
677   displayName: '第11关 鲜果巡礼',
678   fruitIds: levelFruits(11),
679   copiesPerFruit: 6,
680   orderTarget: 3,
681   bufferSize: 5,
682   iceCount: 7,
683   frozenCount: 2,
684   initialVisibleCount: 100,
685   revealPerOrderComplete: 6,
686   plateLanesInitial: 2,
687   ...allTools,
688 },
689 {
690   levelNumber: 12,
691   displayName: '第12关 果香续宴',
692   fruitIds: levelFruits(12),
693   copiesPerFruit: 6,
694   orderTarget: 3,
695   bufferSize: 5,
696   iceCount: 7,
697   frozenCount: 3,
698   initialVisibleCount: 104,
699   revealPerOrderComplete: 6,
700   plateLanesInitial: 2,
```

```
701     ...allTools,
702 },
703 {
704     levelNumber: 13,
705     displayName: '第13关 东方蜜径',
706     fruitIds: levelFruits(13),
707     copiesPerFruit: 6,
708     orderTarget: 3,
709     bufferSize: 5,
710     iceCount: 8,
711     frozenCount: 3,
712     initialVisibleCount: 108,
713     revealPerOrderComplete: 7,
714     plateLanesInitial: 2,
715     ...allTools,
716 },
717 {
718     levelNumber: 14,
719     displayName: '第14关 缤纷拼盘',
720     fruitIds: levelFruits(14),
721     copiesPerFruit: 6,
722     orderTarget: 3,
723     bufferSize: 5,
724     iceCount: 8,
725     frozenCount: 4,
726     initialVisibleCount: 112,
727     revealPerOrderComplete: 7,
728     plateLanesInitial: 2,
729     ...allTools,
730 },
731 {
732     levelNumber: 15,
733     displayName: '第15关 百味协奏',
734     fruitIds: levelFruits(15),
735     copiesPerFruit: 6,
736     orderTarget: 3,
737     bufferSize: 5,
738     iceCount: 9,
739     frozenCount: 4,
740     initialVisibleCount: 116,
741     revealPerOrderComplete: 7,
742     plateLanesInitial: 2,
743     ...allTools,
744 },
745 {
746     levelNumber: 16,
747     displayName: '第16关 十五星上席',
748     fruitIds: levelFruits(16),
749     copiesPerFruit: POST_15_COPIES_PER_FRUIT,
750     orderTarget: 3,
```

```
751     bufferSize: 5,
752     iceCount: 9,
753     frozenCount: 4,
754     initialVisibleCount: 118,
755     revealPerOrderComplete: 7,
756     plateLanesInitial: 2,
757     ...allTools,
758   },
759   {
760     levelNumber: 17,
761     displayName: '第17关 十五星收束',
762     fruitIds: levelFruits(17),
763     copiesPerFruit: POST_15_COPIES_PER_FRUIT,
764     orderTarget: 3,
765     bufferSize: 5,
766     iceCount: 9,
767     frozenCount: 5,
768     initialVisibleCount: 120,
769     revealPerOrderComplete: 7,
770     plateLanesInitial: 2,
771     ...allTools,
772   },
773   {
774     levelNumber: 18,
775     displayName: '第18关 杂味圆舞',
776     fruitIds: levelFruits(18),
777     copiesPerFruit: POST_15_COPIES_PER_FRUIT,
778     orderTarget: 3,
779     bufferSize: 5,
780     iceCount: 10,
781     frozenCount: 5,
782     initialVisibleCount: 122,
783     revealPerOrderComplete: 7,
784     plateLanesInitial: 2,
785     ...allTools,
786   },
787   {
788     levelNumber: 19,
789     displayName: '第19关 薄冰试炼',
790     fruitIds: levelFruits(19),
791     copiesPerFruit: POST_15_COPIES_PER_FRUIT,
792     orderTarget: 3,
793     bufferSize: 5,
794     iceCount: 10,
795     frozenCount: 5,
796     initialVisibleCount: 124,
797     revealPerOrderComplete: 8,
798     plateLanesInitial: 2,
799     ...allTools,
800   },
```



```
801 {
802   levelNumber: 20,
803   displayName: '第20关 四味重奏',
804   fruitIds: levelFruits(20),
805   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
806   orderTarget: 3,
807   bufferSize: 5,
808   iceCount: 11,
809   frozenCount: 6,
810   initialVisibleCount: 126,
811   revealPerOrderComplete: 8,
812   plateLanesInitial: 2,
813   ...allTools,
814 },
815 {
816   levelNumber: 21,
817   displayName: '第21关 果阵初章',
818   fruitIds: levelFruits(21),
819   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
820   orderTarget: 3,
821   bufferSize: 5,
822   iceCount: 11,
823   frozenCount: 6,
824   initialVisibleCount: 128,
825   revealPerOrderComplete: 8,
826   plateLanesInitial: 2,
827   ...allTools,
828 },
829 {
830   levelNumber: 22,
831   displayName: '第22关 果阵回环',
832   fruitIds: levelFruits(22),
833   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
834   orderTarget: 3,
835   bufferSize: 5,
836   iceCount: 12,
837   frozenCount: 6,
838   initialVisibleCount: 130,
839   revealPerOrderComplete: 8,
840   plateLanesInitial: 2,
841   ...allTools,
842 },
843 {
844   levelNumber: 23,
845   displayName: '第23关 滋补雅集',
846   fruitIds: levelFruits(23),
847   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
848   orderTarget: 3,
849   bufferSize: 5,
850   iceCount: 12,
```

```
851   frozenCount: 7,
852   initialVisibleCount: 132,
853   revealPerOrderComplete: 8,
854   plateLanesInitial: 2,
855   ...allTools,
856 },
857 {
858   levelNumber: 24,
859   displayName: '第24关 小料满仓',
860   fruitIds: levelFruits(24),
861   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
862   orderTarget: 3,
863   bufferSize: 5,
864   iceCount: 13,
865   frozenCount: 7,
866   initialVisibleCount: 134,
867   revealPerOrderComplete: 8,
868   plateLanesInitial: 2,
869   ...allTools,
870 },
871 {
872   levelNumber: 25,
873   displayName: '第25关 重味温习',
874   fruitIds: levelFruits(25),
875   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
876   orderTarget: 3,
877   bufferSize: 5,
878   iceCount: 13,
879   frozenCount: 7,
880   initialVisibleCount: 136,
881   revealPerOrderComplete: 9,
882   plateLanesInitial: 2,
883   ...allTools,
884 },
885 {
886   levelNumber: 26,
887   displayName: '第26关 果阵再开',
888   fruitIds: levelFruits(26),
889   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
890   orderTarget: 3,
891   bufferSize: 5,
892   iceCount: 14,
893   frozenCount: 8,
894   initialVisibleCount: 138,
895   revealPerOrderComplete: 9,
896   plateLanesInitial: 2,
897   ...allTools,
898 },
899 {
900   levelNumber: 27,
```

```
901   displayName: '第27关 滋补再炖',
902   fruitIds: levelFruits(27),
903   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
904   orderTarget: 3,
905   bufferSize: 5,
906   iceCount: 14,
907   frozenCount: 8,
908   initialVisibleCount: 140,
909   revealPerOrderComplete: 9,
910   plateLanesInitial: 2,
911   ...allTools,
912 },
913 {
914   levelNumber: 28,
915   displayName: '第28关 小料再添',
916   fruitIds: levelFruits(28),
917   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
918   orderTarget: 3,
919   bufferSize: 5,
920   iceCount: 15,
921   frozenCount: 8,
922   initialVisibleCount: 142,
923   revealPerOrderComplete: 9,
924   plateLanesInitial: 2,
925   ...allTools,
926 },
927 {
928   levelNumber: 29,
929   displayName: '第29关 甘星连珠',
930   fruitIds: levelFruits(29),
931   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
932   orderTarget: 3,
933   bufferSize: 5,
934   iceCount: 15,
935   frozenCount: 9,
936   initialVisibleCount: 144,
937   revealPerOrderComplete: 9,
938   plateLanesInitial: 2,
939   ...allTools,
940 },
941 {
942   levelNumber: 30,
943   displayName: '第30关 廿四终宴',
944   fruitIds: levelFruits(30),
945   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
946   orderTarget: 3,
947   bufferSize: 5,
948   iceCount: 16,
949   frozenCount: 9,
950   initialVisibleCount: 146,
```

```
951   revealPerOrderComplete: 9,
952   plateLanesInitial: 2,
953   ...allTools,
954 },
955 {
956   levelNumber: 31,
957   displayName: '第31关 紫冠初席',
958   fruitIds: levelFruits(31),
959   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
960   orderTarget: 3,
961   bufferSize: 5,
962   iceCount: 16,
963   frozenCount: 9,
964   initialVisibleCount: 148,
965   revealPerOrderComplete: 10,
966   plateLanesInitial: 2,
967   ...allTools,
968 },
969 {
970   levelNumber: 32,
971   displayName: '第32关 油绿奶香',
972   fruitIds: levelFruits(32),
973   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
974   orderTarget: 3,
975   bufferSize: 5,
976   iceCount: 17,
977   frozenCount: 9,
978   initialVisibleCount: 150,
979   revealPerOrderComplete: 10,
980   plateLanesInitial: 2,
981   ...allTools,
982 },
983 {
984   levelNumber: 33,
985   displayName: '第33关 红宝石雨',
986   fruitIds: levelFruits(33),
987   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
988   orderTarget: 3,
989   bufferSize: 5,
990   iceCount: 17,
991   frozenCount: 10,
992   initialVisibleCount: 152,
993   revealPerOrderComplete: 10,
994   plateLanesInitial: 2,
995   ...allTools,
996 },
997 {
998   levelNumber: 34,
999   displayName: '第34关 蜜瓜弯月',
1000  fruitIds: levelFruits(34),
```

```
1001   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1002   orderTarget: 3,
1003   bufferSize: 5,
1004   iceCount: 18,
1005   frozenCount: 10,
1006   initialVisibleCount: 154,
1007   revealPerOrderComplete: 10,
1008   plateLanesInitial: 2,
1009   ...allTools,
1010 },
1011 {
1012   levelNumber: 35,
1013   displayName: '第35关 莲雾粉雾',
1014   fruitIds: levelFruits(35),
1015   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1016   orderTarget: 3,
1017   bufferSize: 5,
1018   iceCount: 18,
1019   frozenCount: 10,
1020   initialVisibleCount: 156,
1021   revealPerOrderComplete: 10,
1022   plateLanesInitial: 2,
1023   ...allTools,
1024 },
1025 {
1026   levelNumber: 36,
1027   displayName: '第36关 回甘青果',
1028   fruitIds: levelFruits(36),
1029   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1030   orderTarget: 3,
1031   bufferSize: 5,
1032   iceCount: 18,
1033   frozenCount: 11,
1034   initialVisibleCount: 158,
1035   revealPerOrderComplete: 10,
1036   plateLanesInitial: 2,
1037   ...allTools,
1038 },
1039 {
1040   levelNumber: 37,
1041   displayName: '第37关 芭乐餐车',
1042   fruitIds: levelFruits(37),
1043   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1044   orderTarget: 3,
1045   bufferSize: 5,
1046   iceCount: 19,
1047   frozenCount: 11,
1048   initialVisibleCount: 160,
1049   revealPerOrderComplete: 11,
1050   plateLanesInitial: 2,
```

```
1051   ...allTools,
1052 },
1053 {
1054   levelNumber: 38,
1055   displayName: '第38关 木瓜暖盅',
1056   fruitIds: levelFruits(38),
1057   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1058   orderTarget: 3,
1059   bufferSize: 5,
1060   iceCount: 19,
1061   frozenCount: 11,
1062   initialVisibleCount: 162,
1063   revealPerOrderComplete: 11,
1064   plateLanesInitial: 2,
1065   ...allTools,
1066 },
1067 {
1068   levelNumber: 39,
1069   displayName: '第39关 无花蜜宴',
1070   fruitIds: levelFruits(39),
1071   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1072   orderTarget: 3,
1073   bufferSize: 5,
1074   iceCount: 20,
1075   frozenCount: 12,
1076   initialVisibleCount: 164,
1077   revealPerOrderComplete: 11,
1078   plateLanesInitial: 2,
1079   ...allTools,
1080 },
1081 {
1082   levelNumber: 40,
1083   displayName: '第40关 杏光终宴',
1084   fruitIds: levelFruits(40),
1085   copiesPerFruit: POST_15_COPIES_PER_FRUIT,
1086   orderTarget: 3,
1087   bufferSize: 5,
1088   iceCount: 20,
1089   frozenCount: 12,
1090   initialVisibleCount: 166,
1091   revealPerOrderComplete: 11,
1092   plateLanesInitial: 2,
1093   ...allTools,
1094 },
1095 ];
1096 export const BOWL_LEVEL_COUNT = BOWL_LEVELS.length;
1097 export function getBowlLevelDef(index: number): BowlLevelDef {
1098   const clamped = Math.max(0, Math.min(index, BOWL_LEVELS.length - 1));
1099   return BOWL_LEVELS[clamped];
1100 }
```

```

1101 /** 相对所有前序关卡新出现的食材（用于过关「解锁食材」展示；第 1 关为全部本关食材） */
1102 export function getNewFruitsIntroducedInLevel(levelIndex: number): FruitId[] {
1103   const def = getBowlLevelDef(levelIndex);
1104   if (levelIndex <= 0) {
1105     return def.fruitIds.slice();
1106   }
1107   const seen = new Set<FruitId>();
1108   for (let i = 0; i < levelIndex; i += 1) {
1109     for (const id of getBowlLevelDef(i).fruitIds) {
1110       seen.add(id);
1111     }
1112   }
1113   return def.fruitIds.filter((id) => !seen.has(id));
1114 }
1115 import { BOWL_IMAGES_ROOT } from '@config/bowlAssets';
1116 export const BOWL_SOUP_KEYS = [
1117   'milk',
1118   'berry_tomato',
1119   'matcha',
1120   'mango_coconut',
1121   'taro_purple',
1122   'cocoa',
1123   'avocado_yogurt',
1124   'pomegranate_soda',
1125 ] as const;
1126 export const BOWL_RIM_KEYS = [
1127   'crystal',
1128   'pink_ceramic',
1129   'mint_glass',
1130   'sesame_clay',
1131   'gold_porcelain',
1132   'star_glass',
1133   'coral_shell',
1134   'honey_amber_glass',
1135 ] as const;
1136 export type BowlSoupKey = (typeof BOWL_SOUP_KEYS)[number];
1137 export type BowlRimKey = (typeof BOWL_RIM_KEYS)[number];
1138 export const DEFAULT_BOWL_SOUP_KEY: BowlSoupKey = 'milk';
1139 export const DEFAULT_BOWL_RIM_KEY: BowlRimKey = 'crystal';
1140 export const BOWL_SOUP_ASSETS: Record<BowlSoupKey, string> = {
1141   milk: `${BOWL_IMAGES_ROOT}/bowl_soup_milk.png`,
1142   berry_tomato: `${BOWL_IMAGES_ROOT}/bowl_soup_berry_tomato.png`,
1143   matcha: `${BOWL_IMAGES_ROOT}/bowl_soup_matcha.png`,
1144   mango_coconut: `${BOWL_IMAGES_ROOT}/bowl_soup_mango_coconut.png`,
1145   taro_purple: `${BOWL_IMAGES_ROOT}/bowl_soup_taro_purple.png`,
1146   cocoa: `${BOWL_IMAGES_ROOT}/bowl_soup_cocoa.png`,
1147   avocado_yogurt: `${BOWL_IMAGES_ROOT}/bowl_soup_avocado_yogurt.png`,
1148   pomegranate_soda: `${BOWL_IMAGES_ROOT}/bowl_soup_pomegranate_soda.png`,
1149 };
1150 export const BOWL_RIM_ASSETS: Record<BowlRimKey, string> = {

```

```

1151 crystal: `${BOWL_IMAGES_ROOT}/bowl_crystal_rim.png`,
1152 pink_ceramic: `${BOWL_IMAGES_ROOT}/bowl_rim_pink_ceramic.png`,
1153 mint_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_mint_glass.png`,
1154 sesame_clay: `${BOWL_IMAGES_ROOT}/bowl_rim_sesame_clay.png`,
1155 gold_porcelain: `${BOWL_IMAGES_ROOT}/bowl_rim_gold_porcelain.png`,
1156 star_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_star_glass.png`,
1157 coral_shell: `${BOWL_IMAGES_ROOT}/bowl_rim_coral_shell.png`,
1158 honey_amber_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_honey_amber_glass.png`,
1159 };
1160 export const BOWL_SOUP_UNLOCKS: Array<{ levelNumber: number; key: BowlSoupKey; label: string }> = [
1161   { levelNumber: 1, key: 'milk', label: '奶白汤' },
1162   { levelNumber: 4, key: 'berry_tomato', label: '草莓番茄红汤' },
1163   { levelNumber: 9, key: 'matcha', label: '抹茶绿汤' },
1164   { levelNumber: 14, key: 'mango_coconut', label: '芒果椰乳黄汤' },
1165   { levelNumber: 20, key: 'taro_purple', label: '芋泥紫汤' },
1166   { levelNumber: 26, key: 'cocoa', label: '可可巧克力汤' },
1167   { levelNumber: 32, key: 'avocado_yogurt', label: '牛油果酸奶绿汤' },
1168   { levelNumber: 36, key: 'pomegranate_soda', label: '石榴气泡红汤' },
1169 ];
1170 export const BOWL_RIM_UNLOCKS: Array<{ levelNumber: number; key: BowlRimKey; label: string }> = [
1171   { levelNumber: 1, key: 'crystal', label: '水晶碗' },
1172   { levelNumber: 6, key: 'pink_ceramic', label: '樱粉陶瓷碗' },
1173   { levelNumber: 11, key: 'mint_glass', label: '青釉玻璃碗' },
1174   { levelNumber: 17, key: 'sesame_clay', label: '黑芝麻陶土碗' },
1175   { levelNumber: 23, key: 'gold_porcelain', label: '金边白瓷碗' },
1176   { levelNumber: 29, key: 'star_glass', label: '星空玻璃碗' },
1177   { levelNumber: 34, key: 'coral_shell', label: '珊瑚贝壳碗' },
1178   { levelNumber: 39, key: 'honey_amber_glass', label: '蜂蜜琥珀玻璃碗' },
1179 ];
1180 export interface BowlSkinUnlock {
1181   kind: 'soup' | 'bowl';
1182   key: BowlSoupKey | BowlRimKey;
1183   label: string;
1184   textureKey: string;
1185 }
1186 export function getBowlSoupKeyForLevel(levelNumber: number): BowlSoupKey {
1187   let key = DEFAULT_BOWL_SOUP_KEY;
1188   for (const unlock of BOWL_SOUP_UNLOCKS) {
1189     if (levelNumber >= unlock.levelNumber) {
1190       key = unlock.key;
1191     }
1192   }
1193   return key;
1194 }
1195 export function getBowlRimKeyForLevel(levelNumber: number): BowlRimKey {
1196   let key = DEFAULT_BOWL_RIM_KEY;
1197   for (const unlock of BOWL_RIM_UNLOCKS) {
1198     if (levelNumber >= unlock.levelNumber) {
1199       key = unlock.key;
1200     }

```



```
1201   }
1202   return key;
1203 }
1204 export function getBowlSkinUnlocksInLevel(levelNumber: number): BowlSkinUnlock[] {
1205   return [
1206     ...BOWL_SOUP_UNLOCKS.filter((unlock) => unlock.levelNumber === levelNumber).map((unlock) => ({
1207       kind: 'soup' as const,
1208       key: unlock.key,
1209       label: unlock.label,
1210       textureKey: `bowl_soup_${unlock.key}`,
1211     })),
1212     ...BOWL_RIM_UNLOCKS.filter((unlock) => unlock.levelNumber === levelNumber).map((unlock) => ({
1213       kind: 'bowl' as const,
1214       key: unlock.key,
1215       label: unlock.label,
1216       textureKey: `bowl_rim_${unlock.key}`,
1217     })),
1218   ];
1219 }
1220 import { BOWL_THEME_IMAGES_ROOT } from '@config/bowlAssets';
1221 export const BOWL_THEME_KEYS = [
1222   'mint_waterpark',
1223   'tropical_fruit_stand',
1224   'pool_ice_drink',
1225   'seaside_night_market',
1226   'garden_picnic',
1227   'fruit_restaurant',
1228   'beach_picnic',
1229 ] as const;
1230 export type BowlThemeKey = (typeof BOWL_THEME_KEYS)[number];
1231 export interface BowlThemeDef {
1232   key: BowlThemeKey;
1233   label: string;
1234   backdropAsset: string;
1235   bgTop: number;
1236   bgBottom: number;
1237   header: number;
1238   headerAccent: number;
1239   board: number;
1240   boardAccent: number;
1241   hudOuter: number;
1242   hudInner: number;
1243   hudStroke: number;
1244   hudText: number;
1245   orderBubble: number;
1246   orderBubbleStroke: number;
1247   progressFill: number;
1248   progressFillHi: number;
1249   slotTint: number;
1250 }
```

```
1251 export const BOWL_THEMES: Record< BowlThemeKey, BowlThemeDef> = {
1252   mint_waterpark: {
1253     key: 'mint_waterpark',
1254     label: '薄荷水上乐园',
1255     backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_mint_waterpark.jpg`,
1256     bgTop: 0xcff9df,
1257     bgBottom: 0xeaffcf,
1258     header: 0x90eadc,
1259     headerAccent: 0x4cc8bd,
1260     board: 0xa8f3df,
1261     boardAccent: 0x60d2c9,
1262     hudOuter: 0x248c91,
1263     hudInner: 0x3bb7b6,
1264     hudStroke: 0xd9fff2,
1265     hudText: 0xf5fff3,
1266     orderBubble: 0xffffee,
1267     orderBubbleStroke: 0x49b8a8,
1268     progressFill: 0x42d8c8,
1269     progressFillHi: 0xbffff2,
1270     slotTint: 0xd7fff5,
1271   },
1272   tropical_fruit_stand: {
1273     key: 'tropical_fruit_stand',
1274     label: '热带果摊',
1275     backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_tropical_fruit_stand.jpg`,
1276     bgTop: 0xffff0b8,
1277     bgBottom: 0xfffffe8,
1278     header: 0xd9a961,
1279     headerAccent: 0x75b85b,
1280     board: 0xe9bd77,
1281     boardAccent: 0xb98343,
1282     hudOuter: 0x5d3c1f,
1283     hudInner: 0x8f5d2f,
1284     hudStroke: 0xffde75,
1285     hudText: 0xffff6bd,
1286     orderBubble: 0xffffef,
1287     orderBubbleStroke: 0x8d6234,
1288     progressFill: 0x71cf65,
1289     progressFillHi: 0xdfffb1,
1290     slotTint: 0xffff1c9,
1291   },
1292   pool_ice_drink: {
1293     key: 'pool_ice_drink',
1294     label: '泳池冰饮',
1295     backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_pool_ice_drink.jpg`,
1296     bgTop: 0xa4f2ff,
1297     bgBottom: 0xe6fff7,
1298     header: 0x7addec,
1299     headerAccent: 0x2ba9d2,
1300     board: 0xb8f4f1,
```

```
1301   boardAccent: 0x58cfdc,
1302   hudOuter: 0x217f9a,
1303   hudInner: 0x3bb5c8,
1304   hudStroke: 0xd9fbff,
1305   hudText: 0xf3ffff,
1306   orderBubble: 0xf7ffff,
1307   orderBubbleStroke: 0x4fbfd4,
1308   progressFill: 0x38c7f1,
1309   progressFillHi: 0xc9f7ff,
1310   slotTint: 0xdbfbff,
1311 },
1312 garden_picnic: {
1313   key: 'garden_picnic',
1314   label: '花园野餐',
1315   backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_garden_picnic.jpg`,
1316   bgTop: 0xffff4e8,
1317   bgBottom: 0xf8fff0,
1318   header: 0xc8e8b0,
1319   headerAccent: 0x8bc98a,
1320   board: 0xe8d8b0,
1321   boardAccent: 0xb8c98a,
1322   hudOuter: 0x5a6b3f,
1323   hudInner: 0x7d8f6a,
1324   hudStroke: 0xffff6d8,
1325   hudText: 0xffff8ef,
1326   orderBubble: 0xfffff2,
1327   orderBubbleStroke: 0x8d9a7a,
1328   progressFill: 0x7fcf8a,
1329   progressFillHi: 0xe8ffd8,
1330   slotTint: 0xffff8ef,
1331 },
1332 seaside_night_market: {
1333   key: 'seaside_night_market',
1334   label: '海边夜市',
1335   backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_seaside_night_market.jpg`,
1336   bgTop: 0x3e426c,
1337   bgBottom: 0xffba89,
1338   header: 0x4b3c67,
1339   headerAccent: 0xffc46b,
1340   board: 0xd8876b,
1341   boardAccent: 0x714e68,
1342   hudOuter: 0x33263e,
1343   hudInner: 0x5b3c61,
1344   hudStroke: 0xffd180,
1345   hudText: 0xffff0c7,
1346   orderBubble: 0xffefd9,
1347   orderBubbleStroke: 0x8d5472,
1348   progressFill: 0xff9f7a,
1349   progressFillHi: 0xffe0a0,
1350   slotTint: 0xffe0d4,
```

```
1351   },
1352   fruit_restaurant: {
1353     key: 'fruit_restaurant',
1354     label: '果味餐厅',
1355     backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_fruit_restaurant.jpg`,
1356     bgTop: 0xffff0e8,
1357     bgBottom: 0xfffff5,
1358     header: 0xe8c4a8,
1359     headerAccent: 0xc98a6a,
1360     board: 0xf0d4b8,
1361     boardAccent: 0xb88868,
1362     hudOuter: 0x6b4030,
1363     hudInner: 0x9a6048,
1364     hudStroke: 0xffe8d0,
1365     hudText: 0xffff8ef,
1366     orderBubble: 0xfffff5,
1367     orderBubbleStroke: 0xa07058,
1368     progressFill: 0xe89070,
1369     progressFillHi: 0xffe0c8,
1370     slotTint: 0xffff0e8,
1371   },
1372   beach_picnic: {
1373     key: 'beach_picnic',
1374     label: '沙滩野餐',
1375     backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_beach_picnic.jpg`,
1376     bgTop: 0xd4f4ff,
1377     bgBottom: 0xffff8e8,
1378     header: 0x8ad4e8,
1379     headerAccent: 0x4ab8d8,
1380     board: 0xc8e8f0,
1381     boardAccent: 0x68b8d0,
1382     hudOuter: 0x2a6888,
1383     hudInner: 0x48a0b8,
1384     hudStroke: 0xe8ffff,
1385     hudText: 0xf5ffff,
1386     orderBubble: 0xf7ffff,
1387     orderBubbleStroke: 0x58a8c8,
1388     progressFill: 0x48c8e8,
1389     progressFillHi: 0xc8f0ff,
1390     slotTint: 0xe8f8ff,
1391   },
1392 };
1393 export const DEFAULT_BOWL_THEME_KEY: BowlThemeKey = 'tropical_fruit_stand';
1394 const THEME_UNLOCKS: Array<{ levelNumber: number; key: BowlThemeKey }> = [
1395   { levelNumber: 1, key: 'tropical_fruit_stand' },
1396   { levelNumber: 4, key: 'mint_waterpark' },
1397   { levelNumber: 7, key: 'pool_ice_drink' },
1398   { levelNumber: 10, key: 'garden_picnic' },
1399   { levelNumber: 13, key: 'seaside_night_market' },
1400   { levelNumber: 16, key: 'fruit_restaurant' },
```

```
1401 { levelNumber: 19, key: 'beach_picnic' },
1402 { levelNumber: 22, key: 'tropical_fruit_stand' },
1403 { levelNumber: 25, key: 'mint_waterpark' },
1404 { levelNumber: 28, key: 'pool_ice_drink' },
1405 { levelNumber: 31, key: 'garden_picnic' },
1406 { levelNumber: 34, key: 'seaside_night_market' },
1407 { levelNumber: 37, key: 'fruit_restaurant' },
1408 { levelNumber: 40, key: 'beach_picnic' },
1409 ];
1410 export function getBowlThemeKeyForLevel(levelNumber: number): BowlThemeKey {
1411   let key = DEFAULT_BOWL_THEME_KEY;
1412   for (const unlock of THEME_UNLOCKS) {
1413     if (levelNumber >= unlock.levelNumber) {
1414       key = unlock.key;
1415     }
1416   }
1417   return key;
1418 }
1419 export function getBowlTheme(key: BowlThemeKey): BowlThemeDef {
1420   return BOWL_THEMES[key] ?? BOWL_THEMES[DEFAULT_BOWL_THEME_KEY];
1421 }
1422 import type { FruitId } from '@config/fruits';
1423 export const DAILY_LIMITED_MAX_FRUIT_TYPES = 19;
1424 export const DAILY_LIMITED_MIN_STACK_CARDS = 210;
1425 /** 每日限定每局每种道具（洗牌 / 撤销 / 抬起）各自可用次数上限 */
1426 export const DAILY_LIMITED_TOOL_USES_PER_ROUND = 3;
1427 export interface DailyDrinkRecipe {
1428   readonly title: string;
1429   readonly intro: string;
1430   readonly steps: readonly string[];
1431 }
1432 export interface DailyThemeTargetDef {
1433   readonly fruitId: FruitId;
1434   readonly requiredCount: number;
1435   readonly cardCopies: number;
1436 }
1437 export interface DailyThemeRecipeCardDef {
1438   readonly textureKey: string;
1439   readonly path: string;
1440   readonly catalogTitle: string;
1441   readonly catalogSubtitle: string;
1442   readonly shareTitle: string;
1443 }
1444 export interface DailyThemeLevelDef {
1445   readonly dayOfMonth: number;
1446   readonly themeId: string;
1447   readonly themeName: string;
1448   readonly drinkName: string;
1449   readonly targets: readonly DailyThemeTargetDef[];
1450 /** 堆叠区总卡数。每日限定必须保持满屏堆叠难度，只能大于等于 DAILY_LIMITED_MIN_STACK_CARDS。 */
```

```
1451   readonly totalStackCards?: number;
1452   readonly fruitIds: readonly FruitId[];
1453   readonly positioningText: string;
1454   readonly recipeUnlock: DailyDrinkRecipe;
1455   readonly recipeCard: DailyThemeRecipeCardDef;
1456   readonly bufferSize: number;
1457   readonly toolCounts: {
1458     readonly shuffle: number;
1459     readonly undo: number;
1460     readonly lift: number;
1461   };
1462   readonly layoutSeed: number;
1463 }
1464 export function getDailyLimitedPlayableFruitIds(level: DailyThemeLevelDef): readonly FruitId[] {
1465   const result: FruitId[] = [];
1466   const seen = new Set<FruitId>();
1467   const push = (fruitId: FruitId): void => {
1468     if (seen.has(fruitId) || result.length >= DAILY_LIMITED_MAX_FRUIT_TYPES) {
1469       return;
1470     }
1471     seen.add(fruitId);
1472     result.push(fruitId);
1473   };
1474   // 目标水果必须优先保留; 剩余名额再补干扰水果, 保证实际发牌种类不超过 19。
1475   for (const target of level.targets) {
1476     push(target.fruitId);
1477   }
1478   for (const fruitId of level.fruitIds) {
1479     push(fruitId);
1480   }
1481   return result;
1482 }
1483 const DAILY_LIMITED_COMMON_FRUITS: readonly FruitId[] = [
1484   'pineapple',
1485   'apple',
1486   'avocado',
1487   'banana',
1488   'bayberry',
1489   'blueberry',
1490   'cantaloupe',
1491   'cherry_tomato',
1492   'cucumber',
1493   'dragonfruit',
1494   'emblic',
1495   'grape_green',
1496   'grapefruit',
1497   'guava',
1498   'kumquat',
1499   'lemon',
1500   'lily_bulb',
```

```
30744   session_key: typeof data.session_key === 'string' ? data.session_key : ",
30745   };
30746 }
30747 async function ttCode2Openid(code) {
30748   const appid = getPlatformCredential('tt', 'APPID');
30749   const secret = getPlatformCredential('tt', 'SECRET');
30750   if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...
30751   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
30752   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
30753   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
30754     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
30755   }
30756   return data.data.openid;
30757 }
30758 function httpGetJson(url) {
30759   return httpRequestJson(url, 'GET');
30760 }
30761 function httpPostJson(url, body) {
30762   return httpRequestJson(url, 'POST', body);
30763 }
30764 function httpRequestJson(url, method, body) {
30765   if (typeof fetch !== 'function') {
30766     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
30767   }
30768   const opts = { method, headers: { 'Content-Type': 'application/json' } };
30769   if (body !== undefined) {
30770     opts.body = JSON.stringify(body);
30771   }
30772   return fetch(url, opts).then(async (res) => {
30773     const text = await res.text();
30774     try {
30775       return JSON.parse(text);
30776     } catch (_) {
30777       return { _raw: text };
30778     }
30779   });
30780 }
30781 module.exports = {
30782   handleLogin,
30783   requireUser,
30784 };
30785 const DEFAULT_GAME_KEY = 'hotpot';
30786 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
30787 const DEFAULT_MAX_BYTES = 256 * 1024;
30788 const DEFAULT_RANK_LIST_LIMIT = 50;
30789 const DEFAULT_RANK_LIST_MAX_LIMIT = 100;
30790 const DEFAULT_RANK_MINE_SCAN_LIMIT = 1000;
30791 const DEFAULT_RANK_BOWL_MAX_LEVEL = 40;
30792 const DEFAULT_RANK_FRUIT_MAX_SCORE = 9999999;
30793 const DEFAULT_LEVEL_PASS_RATE_COLLECTION = 'hotpot_public_level_pass_rates';
```

```
30794 function getGameKey() {
30795   const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
30796   if (!v) return DEFAULT_GAME_KEY;
30797   if (!/^[a-z][a-z0-9_-]{0,31}$/.test(v)) {
30798     throw new Error(`非法 GAME_KEY: ${v}`);
30799   }
30800   return v;
30801 }
30802 function gameKeyUpper() {
30803   return getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
30804 }
30805 function readEnvPrefer(...keys) {
30806   for (const key of keys) {
30807     const value = process.env[key];
30808     if (value !== undefined && value !== null && String(value).length > 0) {
30809       return String(value);
30810     }
30811   }
30812   return '';
30813 }
30814 function getCollectionName(suffix) {
30815   const overrideKey = `${gameKeyUpper()}_${suffix.toUpperCase()}_COLLECTION`;
30816   const override = process.env[overrideKey];
30817   if (override) {
30818     return String(override);
30819   }
30820   return `${getGameKey()}_${suffix}`;
30821 }
30822 function getJwtSecret() {
30823   return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
30824 }
30825 function getTtlSec() {
30826   const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
30827   const v = Number(raw);
30828   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
30829 }
30830 function getMaxBytes() {
30831   const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
30832   const v = Number(raw);
30833   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
30834 }
30835 function getPlatformCredential(platform, field) {
30836   const upper = gameKeyUpper();
30837   const platformUpper = platform.toUpperCase();
30838   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
30839 }
30840 function readPositiveIntEnv(key, fallback) {
30841   const v = Number(readEnvPrefer(`${gameKeyUpper()}_${key}`));
30842   return Number.isFinite(v) && v > 0 ? Math.floor(v) : fallback;
30843 }
```



```
30844 function getRankListLimit() {
30845   return readPositiveIntEnv('RANK_LIST_LIMIT', DEFAULT_RANK_LIST_LIMIT);
30846 }
30847 function getRankListMaxLimit() {
30848   return readPositiveIntEnv('RANK_LIST_MAX_LIMIT', DEFAULT_RANK_LIST_MAX_LIMIT);
30849 }
30850 function getRankMineScanLimit() {
30851   return readPositiveIntEnv('RANK_MINE_SCAN_LIMIT', DEFAULT_RANK_MINE_SCAN_LIMIT);
30852 }
30853 function getRankBowlMaxLevel() {
30854   return readPositiveIntEnv('RANK_BOWL_MAX_LEVEL', DEFAULT_RANK_BOWL_MAX_LEVEL);
30855 }
30856 function getRankFruitMaxScore() {
30857   return readPositiveIntEnv('RANK_FRUIT_MAX_SCORE', DEFAULT_RANK_FRUIT_MAX_SCORE);
30858 }
30859 function getLevelPassRateCollectionName() {
30860   return readEnvPrefer(`${gameKeyUpper()}_LEVEL_PASS_RATE_COLLECTION`) || DEFAULT_LEVEL_PASS_RATE...
30861 }
30862 module.exports = {
30863   getGameKey,
30864   gameKeyUpper,
30865   getCollectionName,
30866   getJwtSecret,
30867   getTtlSec,
30868   getMaxBytes,
30869   getPlatformCredential,
30870   getRankListLimit,
30871   getRankListMaxLimit,
30872   getRankMineScanLimit,
30873   getRankBowlMaxLevel,
30874   getRankFruitMaxScore,
30875   getLevelPassRateCollectionName,
30876 };
30877 const tcb = require('@cloudbase/node-sdk');
30878 const { getCollectionName, getLevelPassRateCollectionName } = require('./config');
30879 let app = null;
30880 function getApp() {
30881   if (app) return app;
30882   app = tcb.init({
30883     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
30884   });
30885   return app;
30886 }
30887 function getDb() {
30888   return getApp().database();
30889 }
30890 function getCollection() {
30891   return getDb().collection(getCollectionName('playerData'));
30892 }
30893 function getRankingCollection() {
```

```
30894 return getDb().collection(getCollectionName('rankings'));
30895 }
30896 function getLevelPassRateCollection() {
30897   return getDb().collection(getLevelPassRateCollectionName());
30898 }
30899 module.exports = {
30900   getApp,
30901   getDb,
30902   getCollection,
30903   getRankingCollection,
30904   getLevelPassRateCollection,
30905 };
30906 const crypto = require('crypto');
30907 const { httpError } = require('./http');
30908 const { getDb } = require('./db');
30909 const { getCollectionName } = require('./config');
30910 function getSessionCollection() {
30911   return getDb().collection(getCollectionName('wxSessions'));
30912 }
30913 async function upsertWxSession(userId, sessionKey) {
30914   if (!sessionKey) {
30915     return;
30916   }
30917   const col = getSessionCollection();
30918   const now = Date.now();
30919   const existingRes = await col.where({ userId }).limit(1).get();
30920   const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
30921   if (existing && existing._id) {
30922     await col.doc(existing._id).update({
30923       sessionKey,
30924       updatedAt: now,
30925     });
30926     return;
30927   }
30928   await col.add({
30929     userId,
30930     sessionKey,
30931     updatedAt: now,
30932   });
30933 }
30934 async function readWxSessionKey(userId) {
30935   const col = getSessionCollection();
30936   const res = await col.where({ userId }).limit(1).get();
30937   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
30938   return doc && typeof doc.sessionKey === 'string' ? doc.sessionKey : '';
30939 }
30940 function decryptGameClubPayload(sessionKey, encryptedData, iv) {
30941   if (!sessionKey || !encryptedData || !iv) {
30942     throw httpError(400, 'BAD_GAME_CLUB_PAYLOAD', 'encryptedData / iv / sessionKey 缺失');
30943   }
```

```
30944   try {
30945     const decipher = crypto.createDecipheriv(
30946       'aes-128-cbc',
30947       Buffer.from(sessionKey, 'base64'),
30948       Buffer.from(iv, 'base64'),
30949     );
30950     decipher.setAutoPadding(true);
30951     const decoded = Buffer.concat([
30952       decipher.update(Buffer.from(encryptedData, 'base64')),
30953       decipher.final(),
30954     ]).toString('utf8');
30955     return JSON.parse(decoded);
30956   } catch (error) {
30957     throw httpError(400, 'DECRYPT_FAIL', error && error.message ? error.message : '游戏圈数据解密失败');
30958   }
30959 }
30960 async function handleDecrypt(req) {
30961   // Lazy-load to avoid a circular dependency with auth.js, which stores wx sessions here.
30962   const { requireUser } = require('./auth');
30963   const { userId } = requireUser(req);
30964   const body = req.body || {};
30965   const encryptedData = String(body.encryptedData || '').trim();
30966   const iv = String(body.iv || '').trim();
30967   const sessionKey = await readWxSessionKey(userId);
30968   if (!sessionKey) {
30969     throw httpError(400, 'NO_WX_SESSION', '微信 session 未就绪, 请重新进入游戏');
30970   }
30971   return decryptGameClubPayload(sessionKey, encryptedData, iv);
30972 }
30973 module.exports = {
30974   upsertWxSession,
30975   handleDecrypt,
30976 };
30977 const { getGameKey } = require('./config');
30978 const CORS_HEADERS = {
30979   'Access-Control-Allow-Origin': '*',
30980   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
30981   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
30982   'Access-Control-Max-Age': '86400',
30983 };
30984 function respond(statusCode, body, extraHeaders = {}) {
30985   return {
30986     statusCode,
30987     headers: {
30988       'Content-Type': 'application/json; charset=utf-8',
30989       ...CORS_HEADERS,
30990       ...extraHeaders,
30991     },
30992     body: typeof body === 'string' ? body : JSON.stringify(body),
30993     isBase64Encoded: false,
```

```
30994   };
30995 }
30996 function preflight() {
30997   return {
30998     statusCode: 204,
30999     headers: { ...CORS_HEADERS },
31000     body: "",
31001     isBase64Encoded: false,
31002   };
31003 }
31004 function parseEvent(event) {
31005   event = event || {};
31006   if (event.httpMethod) {
31007     const method = String(event.httpMethod).toUpperCase();
31008     let rawBody = event.body || "";
31009     if (event.isBase64Encoded && rawBody) {
31010       try {
31011         rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
31012       } catch (_) {}
31013     }
31014     let body = {};
31015     if (rawBody) {
31016       try {
31017         body = JSON.parse(rawBody);
31018       } catch (_) {}
31019       body = {};
31020     }
31021   }
31022   return {
31023     method,
31024     path: normalizePath(event.path || '/'),
31025     body,
31026     headers: lowercaseHeaders(event.headers || {}),
31027     query: event.queryStringParameters || {},
31028     raw: event,
31029   };
31030 }
31031 const action = String(event.action || "").replace(/^\//+/, "");
31032 return {
31033   method: 'POST',
31034   path: action ? `/${action}` : '/',
31035   body: event.body || {},
31036   headers: lowercaseHeaders(event.headers || {}),
31037   query: {},
31038   raw: event,
31039 };
31040 }
31041 function normalizePath(path) {
31042   if (!path) return '/';
31043   let p = String(path);
```

```
31044 if (!p.startsWith('/')) p = `/${p}`;
31045 const apiPrefix = `${getGameKey()}-api`;
31046 p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$)`), '');
31047 if (p === '') p = '/';
31048 if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
31049 return p;
31050 }
31051 function lowercaseHeaders(headers) {
31052   const out = {};
31053   for (const key of Object.keys(headers || {})) {
31054     out[key.toLowerCase()] = headers[key];
31055   }
31056   return out;
31057 }
31058 function httpError(status, code, message) {
31059   const err = new Error(message || code);
31060   err.status = status;
31061   err.code = code;
31062   return err;
31063 }
31064 function escapeRegExp(text) {
31065   return String(text).replace(/[*+?^$ {} () [\] \\\]/g, '\\$&');
31066 }
31067 module.exports = {
31068   respond,
31069   preflight,
31070   parseEvent,
31071   httpError,
31072 };
31073 const { requireUser } = require('./auth');
31074 const { getLevelPassRateCollection } = require('./db');
31075 const { httpError } = require('./http');
31076 const MODE_BOWL = 'bowl';
31077 const WINDOW_DAYS = 30;
31078 const DOC_ID = `${MODE_BOWL}_${WINDOW_DAYS}d_latest`;
31079 function normalizeMode(value) {
31080   const mode = String(value || MODE_BOWL).trim();
31081   if (mode !== MODE_BOWL) {
31082     throw httpError(400, 'BAD_MODE', `unsupported mode_key: ${mode}`);
31083   }
31084   return mode;
31085 }
31086 function normalizeWindowDays(value) {
31087   const days = Number(value || WINDOW_DAYS);
31088   if (!Number.isFinite(days) || Math.floor(days) !== WINDOW_DAYS) {
31089     throw httpError(400, 'BAD_WINDOW_DAYS', `unsupported window_days: ${value}`);
31090   }
31091   return WINDOW_DAYS;
31092 }
31093 function publicSnapshot(doc) {
```

```
31094 return {
31095   game_key: String(doc.game_key || 'hotpot'),
31096   mode_key: MODE_BOWL,
31097   window_days: WINDOW_DAYS,
31098   window_start_date: String(doc.window_start_date || ''),
31099   window_end_date: String(doc.window_end_date || ''),
31100   computed_at: Number(doc.computed_at) || 0,
31101   levels: Array.isArray(doc.levels) ? doc.levels.map(publicLevel).filter(Boolean) : [],
31102 };
31103 }
31104 function publicLevel(item) {
31105   const levelId = Number(item && item.level_id);
31106   if (!Number.isFinite(levelId) || levelId <= 0) {
31107     return null;
31108   }
31109   const passRate = Number(item.pass_rate);
31110   return {
31111     level_id: Math.floor(levelId),
31112     pass_rate: Number.isFinite(passRate) ? Math.max(0, Math.min(1, passRate)) : 0,
31113     start_users: Math.max(0, Math.floor(Number(item.start_users) || 0)),
31114     clear_users: Math.max(0, Math.floor(Number(item.clear_users) || 0)),
31115     started_and_cleared_users: Math.max(0, Math.floor(Number(item.started_and_cleared_users) || 0)),
31116     is_sample_low: item.is_sample_low === true || item.is_sample_low === 1,
31117   };
31118 }
31119 async function handleList(req) {
31120   requireUser(req);
31121   const body = req.body || {};
31122   normalizeMode(body.mode_key);
31123   normalizeWindowDays(body.window_days);
31124   const res = await getLevelPassRateCollection().doc(DOC_ID).get();
31125   const doc = res && (Array.isArray(res.data) ? res.data[0] : res.data);
31126   if (!doc) {
31127     throw httpError(404, 'LEVEL_PASS_RATE_NOT_READY', '关卡通关率快照尚未生成');
31128   }
31129   return publicSnapshot(doc);
31130 }
31131 module.exports = {
31132   handleList,
31133 };
31134 const crypto = require('crypto');
31135 const { requireUser } = require('./auth');
31136 const { getRankingCollection } = require('./db');
31137 const { httpError } = require('./http');
31138 const {
31139   getRankBowlMaxLevel,
31140   getRankFruitMaxScore,
31141   getRankListLimit,
31142   getRankListMaxLimit,
31143   getRankMineScanLimit,
```

```
31144 } = require('./config');
31145 const BOARD_BOWL = 'bowl_progress';
31146 const BOARD_FRUIT = 'fruit_best';
31147 const BOARDS = new Set([BOARD_BOWL, BOARD_FRUIT]);
31148 const RANK_SUBMIT_BLOCKED_USER_IDS = new Set([
31149   // GM 测试账号：不再写入 / 更新排行榜数据，避免测试进度污染线上榜单。
31150   'wx:oB0xx3SeJgkkU0_ONokPrzvFljrE',
31151 ]);
31152 function normalizeBoard(value) {
31153   const board = String(value || '').trim();
31154   if (!BOARDS.has(board)) {
31155     throw httpError(400, 'BAD_BOARD', `unsupported board: ${board}`);
31156   }
31157   return board;
31158 }
31159 function normalizeNonNegativeInt(value, field) {
31160   const n = Number(value);
31161   if (!Number.isFinite(n) || n < 0) {
31162     throw httpError(400, `BAD_${field.toUpperCase()}`, `${field} 必须为非负整数`);
31163   }
31164   return Math.floor(n);
31165 }
31166 function normalizeLimit(value) {
31167   const fallback = getRankListLimit();
31168   const max = Math.max(1, getRankListMaxLimit());
31169   const n = Number(value);
31170   if (!Number.isFinite(n) || n <= 0) {
31171     return Math.min(fallback, max);
31172   }
31173   return Math.min(Math.floor(n), max);
31174 }
31175 function normalizeOffset(value) {
31176   const n = Number(value);
31177   return Number.isFinite(n) && n > 0 ? Math.floor(n) : 0;
31178 }
31179 function displayNameForUser(userId) {
31180   const digest = crypto.createHash('sha1').update(String(userId)).digest('hex');
31181   const suffix = String(parseInt(digest.slice(0, 8), 16) % 10000).padStart(4, '0');
31182   return `水果达人${suffix}`;
31183 }
31184 function sanitizeDisplayName(value, fallback) {
31185   const text = String(value || '').trim().replace(/[\r\n\t]/g, ' ').slice(0, 16);
31186   return text || fallback;
31187 }
31188 /**
31189  * 仅接受 http(s):// 开头、长度不超过 1024 的头像 URL；
31190  * 其余视为无效，回退到 fallback（一般是已有头像或空字符串）。
31191  */
31192 function sanitizeAvatarUrl(value, fallback) {
31193   const text = String(value || '').trim();
```

```
31194   if (!text) {
31195     return fallback || "";
31196   }
31197   if (text.length > 1024) {
31198     return fallback || "";
31199   }
31200   if (!/^https?:\/\//.i.test(text)) {
31201     return fallback || "";
31202   }
31203   return text;
31204 }
31205 function isBetterRecord(board, next, prev) {
31206   if (!prev) return true;
31207   if (board === BOARD_BOWL) {
31208     const prevBadge = bowlClearedLevel(prev);
31209     const nextBadge = Number(next.badgeLevel) || 0;
31210     return nextBadge > prevBadge;
31211   }
31212   return next.score > (Number(prev.score) || 0);
31213 }
31214 function bowlClearedLevel(doc) {
31215   if (!doc) return 0;
31216   if (doc.badgeLevel !== undefined && doc.badgeLevel !== null) {
31217     return Number(doc.badgeLevel) || 0;
31218   }
31219   // 极旧记录如果没有 badgeLevel, 则回退到 level, 避免真实上榜记录消失。
31220   return Number(doc.level) || 0;
31221 }
31222 function publicRecord(doc, userId, rank) {
31223   if (!doc) return null;
31224   const out = {
31225     rank: rank || null,
31226     board: doc.board,
31227     displayName: doc.displayName || displayNameForUser(doc.userId || ""),
31228     avatarUrl: doc.avatarUrl || "",
31229     isMe: !!userId && doc.userId === userId,
31230     updatedAt: Number(doc.updatedAt) || 0,
31231   };
31232   if (doc.board === BOARD_BOWL) {
31233     const clearedLevel = bowlClearedLevel(doc);
31234     out.level = clearedLevel;
31235     out.badgeLevel = clearedLevel;
31236   } else {
31237     out.score = Number(doc.score) || 0;
31238   }
31239   return out;
31240 }
31241 function orderedQuery(col, board) {
31242   let query = col.where({ board });
31243   if (board === BOARD_BOWL) {
```



```
31244   query = query.orderBy('badgeLevel', 'desc').orderBy('updatedAt', 'asc');
31245 } else {
31246   query = query.orderBy('score', 'desc').orderBy('updatedAt', 'asc');
31247 }
31248 return query;
31249 }
31250 async function findMineRank(col, board, userId) {
31251   const scanLimit = Math.max(getRankListMaxLimit(), getRankMineScanLimit());
31252   const res = await orderedQuery(col, board).limit(scanLimit).get();
31253   const list = (res && Array.isArray(res.data) ? res.data : []).filter(
31254     (item) => board !== BOARD_BOWL || bowlClearedLevel(item) > 0,
31255   );
31256   const index = list.findIndex((item) => item.userId === userId);
31257   if (index < 0) {
31258     return null;
31259   }
31260   return {
31261     rank: index + 1,
31262     doc: list[index],
31263   };
31264 }
31265 function buildSubmitRecord(board, body) {
31266   if (board === BOARD_BOWL) {
31267     const badgeLevel = normalizeNonNegativeInt(body.badgeLevel ?? body.level ?? 0, 'badgeLevel');
31268     const maxLevel = getRankBowlMaxLevel();
31269     if (badgeLevel > maxLevel) {
31270       throw httpError(400, 'LEVEL_TOO_HIGH', `badgeLevel 超出上限: ${badgeLevel} > ${maxLevel}`);
31271     }
31272     // level 与 badgeLevel 统一为「已通关关数」，兼容旧客户端仍传 level 的情况
31273     return { level: badgeLevel, badgeLevel, score: 0 };
31274   }
31275   const score = normalizeNonNegativeInt(body.score, 'score');
31276   const maxScore = getRankFruitMaxScore();
31277   if (score > maxScore) {
31278     throw httpError(400, 'SCORE_TOO_HIGH', `score 超出上限: ${score} > ${maxScore}`);
31279   }
31280   return { score, level: 0, badgeLevel: 0 };
31281 }
31282 async function handleSubmit(req) {
31283   const { userId, platform } = requireUser(req);
31284   const body = req.body || {};
31285   const board = normalizeBoard(body.board);
31286   const incoming = buildSubmitRecord(board, body);
31287   if (board === BOARD_BOWL && incoming.badgeLevel <= 0) {
31288     return {
31289       board,
31290       updated: false,
31291       reason: 'NO_PROGRESS',
31292       record: null,
31293     };
31294   }
```

```
31294 }
31295 if (RANK_SUBMIT_BLOCKED_USER_IDS.has(userId)) {
31296   console.log(`[rank.submit] uid=${userId} board=${board} blocked GM test account`);
31297   return {
31298     board,
31299     updated: false,
31300     reason: 'GM_TEST_ACCOUNT',
31301     record: null,
31302   };
31303 }
31304 const col = getRankingCollection();
31305 const existingRes = await col.where({ userId, board }).limit(1).get();
31306 const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
31307 const now = Date.now();
31308 const displayName = sanitizeDisplayName(body.displayName, (existing && existing.displayName) || ...
31309 const avatarUrl = sanitizeAvatarUrl(body.avatarUrl, (existing && existing.avatarUrl) || "");
31310 // 排错关键日志: 客户端真的把昵称/头像传上来了吗? sanitize 之后还剩什么?
31311 const rawAvatar = String(body.avatarUrl || "");
31312 console.log(
31313   `[rank.submit] uid=${userId} board=${board}` +
31314   ` rawDisplayName="${body.displayName || ""}" -> "${displayName}"` +
31315   ` rawAvatarUrl=${rawAvatar ? rawAvatar.slice(0, 48) + (rawAvatar.length > 48 ? '...': '') ...` +
31316   ` sanitizedAvatar=${avatarUrl ? 'yes': 'no'}` +
31317   ` hasExisting=${!!existing}`,
31318 );
31319 if (!isBetterRecord(board, incoming, existing)) {
31320   // 即使成绩不再更优, 玩家也可能刚授权了新的微信昵称 / 头像 --
31321   // 这种情况下做轻量字段更新即可, 不动 score/level/badge。
31322   if (existing && existing._id) {
31323     const sameName = existing.displayName === displayName;
31324     const sameAvatar = (existing.avatarUrl || "") === avatarUrl;
31325     if (!sameName || !sameAvatar) {
31326       const patch = { displayName, avatarUrl, updatedAt: now };
31327       await col.doc(existing._id).update(patch);
31328       console.log(
31329         `[rank.submit] uid=${userId} profile_update` +
31330         ` nameChanged=${!sameName} avatarChanged=${!sameAvatar}`,
31331       );
31332       return {
31333         board,
31334         updated: true,
31335         mode: 'profile_update',
31336         record: publicRecord({ ...existing, ...patch }, userId, null),
31337       };
31338     }
31339     console.log(`[rank.submit] uid=${userId} NOT_BETTER, profile already up to date`);
31340   } else {
31341     console.log(`[rank.submit] uid=${userId} NOT_BETTER, no existing doc`);
31342   }
31343   return {
```

```
31344   board,
31345   updated: false,
31346   reason: 'NOT_BETTER',
31347   record: publicRecord(existing, userId, null),
31348   };
31349 }
31350 const docData = {
31351   userId,
31352   board,
31353   platform,
31354   displayName,
31355   avatarUrl,
31356   score: incoming.score,
31357   level: incoming.level,
31358   badgeLevel: incoming.badgeLevel,
31359   createdAt: (existing && existing.createdAt) || now,
31360   updatedAt: now,
31361 };
31362 if (existing && existing._id) {
31363   await col.doc(existing._id).update(docData);
31364   console.log(`[rank.submit] uid=${userId} update score-record id=${existing._id}`);
31365   return { board, updated: true, mode: 'update', record: publicRecord({ ...existing, ...docData...
31366 }
31367 const addRes = await col.add(docData);
31368 console.log(`[rank.submit] uid=${userId} insert new score-record id=${addRes.id} && (addRes.id || a...
31369 return {
31370   board,
31371   updated: true,
31372   mode: 'insert',
31373   record: publicRecord({ ...docData, _id: addRes.id && (addRes.id || addRes._id) }, userId, null),
31374 };
31375 }
31376 async function handleList(req) {
31377   const { userId } = requireUser(req);
31378   const body = req.body || {};
31379   const board = normalizeBoard(body.board);
31380   const limit = normalizeLimit(body.limit);
31381   const offset = normalizeOffset(body.offset);
31382   const col = getRankingCollection();
31383   const res = await orderedQuery(col, board).skip(offset).limit(limit).get();
31384   const rows = (res && Array.isArray(res.data) ? res.data : []).filter(
31385     (doc) => board !== BOARD_BOWL || bowlClearedLevel(doc) > 0,
31386   );
31387   const list = rows.map((doc, index) => publicRecord(doc, userId, offset + index + 1));
31388   const mineRank = await findMineRank(col, board, userId);
31389   const mine = mineRank ? publicRecord(mineRank.doc, userId, mineRank.rank) : null;
31390   return { board, limit, offset, list, mine };
31391 }
31392 async function handleMine(req) {
31393   const { userId } = requireUser(req);
```

```
31394 const board = normalizeBoard((req.body || {}).board);
31395 const col = getRankingCollection();
31396 const mineRank = await findMineRank(col, board, userId);
31397 return { board, mine: mineRank ? publicRecord(mineRank.doc, userId, mineRank.rank) : null };
31398 }
31399 module.exports = {
31400   BOARD_BOWL,
31401   BOARD_FRUIT,
31402   handleSubmit,
31403   handleList,
31404   handleMine,
31405 };
31406 const { httpError } = require('./http');
31407 const { requireUser } = require('./auth');
31408 const { getCollection } = require('./db');
31409 const { getMaxBytes } = require('./config');
31410 async function handlePull(req) {
31411   const { userId, platform } = requireUser(req);
31412   const col = getCollection();
31413   const res = await col.where({ userId }).limit(1).get();
31414   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
31415   if (!doc) {
31416     return {
31417       userId,
31418       platform,
31419       exists: false,
31420       schemaVersion: 0,
31421       updatedAt: 0,
31422       payload: {},
31423       payloadKeys: [],
31424     };
31425   }
31426   return {
31427     userId,
31428     platform,
31429     exists: true,
31430     schemaVersion: doc.schemaVersion || 0,
31431     updatedAt: doc.updatedAt || 0,
31432     payload: doc.payload || {},
31433     payloadKeys: Array.isArray(doc.payloadKeys) ? doc.payloadKeys : Object.keys(doc.payload || {}),
31434     clientFingerprint: doc.clientFingerprint || "",
31435   };
31436 }
31437 async function handlePush(req) {
31438   const { userId, platform } = requireUser(req);
31439   const body = req.body || {};
31440   const schemaVersion = Number(body.schemaVersion);
31441   const updatedAt = Number(body.updatedAt);
31442   const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
31443   const payload = body.payload;
```

```
31444 const clientFingerprint = String(body.clientFingerprint || "").slice(0, 200);
31445 const force = body.force === true;
31446 if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
31447   throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
31448 }
31449 if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
31450   throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
31451 }
31452 if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
31453   throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
31454 }
31455 if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
31456   throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
31457 }
31458 const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
31459 const maxBytes = getMaxBytes();
31460 if (size > maxBytes) {
31461   throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
31462 }
31463 const payloadKeys = [];
31464 for (const key of Object.keys(payload)) {
31465   if (typeof payload[key] !== 'string') {
31466     throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${key}] 必须是字符串`);
31467   }
31468   payloadKeys.push(key);
31469 }
31470 const col = getCollection();
31471 const existingRes = await col.where({ userId }).limit(1).get();
31472 const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
31473 if (existing && !force) {
31474   const prevUpdatedAt = Number(existing.updatedAt) || 0;
31475   if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
31476     throw Object.assign(
31477       httpError(409, 'STALE_UPDATE', `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, b...
31478     {
31479       data: {
31480         remote: {
31481           schemaVersion: existing.schemaVersion || 0,
31482           updatedAt: prevUpdatedAt,
31483           payload: existing.payload || {},
31484           payloadKeys: Array.isArray(existing.payloadKeys)
31485             ? existing.payloadKeys
31486             : Object.keys(existing.payload || {}),
31487         },
31488       },
31489     },
31490   );
31491   }
31492 }
31493 const now = Date.now();
```

```

31494   const docData = {
31495     userId,
31496     platform,
31497     schemaVersion,
31498     updatedAt,
31499     baseRemoteUpdatedAt,
31500     clientFingerprint,
31501     payload,
31502     payloadKeys,
31503     lastWriteAt: now,
31504   };
31505   if (existing && existing._id) {
31506     await col.doc(existing._id).update(docData);
31507     return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: size };
31508   }
31509   const addRes = await col.add(docData);
31510   return {
31511     userId,
31512     updatedAt,
31513     savedAt: now,
31514     mode: 'insert',
31515     sizeBytes: size,
31516     _id: addRes && (addRes.id || addRes._id),
31517   };
31518 }
31519 module.exports = {
31520   handlePull,
31521   handlePush,
31522 };
31523 /**
31524  * 别捞水果好友榜 openDataContext 入口
31525  * -----
31526  * 运行在独立 JS 上下文（WeChat Mini-Game openDataContext 沙箱）中，
31527  * 只能访问受限 wx API:
31528  *   - wx.getSharedCanvas()   获取共享 canvas（主域通过 PIXI Sprite 上屏）
31529  *   - wx.onMessage(cb)      接收主域发来的渲染参数
31530  *   - wx.getFriendCloudStorage() 拉好友（含自己）的 KVDataList
31531  *
31532  * 主域消息协议（见 src/utils/friendRanking.ts）：
31533  *   { action: 'render', tab:'bowl'|'fruit', pixelRatio, width, height, scrollY, selfOpenId, for...
31534  *
31535  * UI 风格说明（重要）：
31536  *   微信不允许主域获取好友明文，绘制必须在子域 canvas API 上完成。
31537  *   为了让好友榜跟主域的世界榜（src/scenes/LeaderboardScene.ts 中 createRankRow）
31538  *   看起来完全一致，本文件里的颜色、行高、字号、圆角等全部按主域同名常量 1:1 复刻。
31539  */
31540 /* eslint-disable */
31541 var CANVAS = null;
31542 var CTX = null;
31543 try {

```

```
31544 CANVAS = wx.getSharedCanvas();
31545 CTX = CANVAS.getContext('2d');
31546 } catch (e) {
31547 // 非小游戏环境（例如开发工具预览），无 sharedCanvas
31548 }
31549 // 跟主域 src/utils/friendRanking.ts:KV_KEY 保持一致
31550 var TAB_META = {
31551   bowl: { key: 'hotpot_bowl_level', unit: '关' },
31552   fruit: { key: 'hotpot_fruit_score', unit: '分' },
31553 };
31554 var MEDAL_ASSETS = {
31555   1: 'assets/images/leaderboard_medal_1.png',
31556   2: 'assets/images/leaderboard_medal_2.png',
31557   3: 'assets/images/leaderboard_medal_3.png',
31558 };
31559 var state = {
31560   tab: 'bowl',
31561   pixelRatio: 2,
31562   scrollY: 0,
31563   selfOpenId: '',
31564   listCache: {},
31565   loading: false,
31566   avatarImgs: {},
31567   medallImgs: {},
31568   pendingRender: null,
31569   inflight: {},
31570 };
31571 var CACHE_TTL_MS = 60 * 1000;
31572 var FONT_FALLBACK = ["PingFang SC", "Microsoft YaHei", "Helvetica Neue", "sans-serif"];
31573 // 与主域 LeaderboardScene 内同名常量对齐
31574 var COLOR = {
31575   CARD_WHITE: '#fdf6ec',
31576   CARD_STROKE: '#eadbc5',
31577   ROW_BG: '#fff9ee',
31578   ROW_STROKE: '#e9d8b9',
31579   ME_BG: '#ffa743',
31580   ME_STROKE: '#d6791f',
31581   TEXT_DARK: '#5a3318',
31582   TEXT_LIGHT: '#ffffff',
31583   TEXT_ME_STROKE: '#a14400',
31584   TEXT_RANK_NUM: '#9b8268',
31585   MEDAL_GOLD_CORE: '#f7c64a',
31586   MEDAL_GOLD_EDGE: '#c88517',
31587   MEDAL_SILVER_CORE: '#d8e2ec',
31588   MEDAL_SILVER_EDGE: '#8aa3b9',
31589   MEDAL_BRONZE_CORE: '#e79768',
31590   MEDAL_BRONZE_EDGE: '#a15a2a',
31591   MEDAL_RIBBON: '#d94b4b',
31592   HINT_TEXT: '#8a5a2b',
31593   HINT_SUB: '#b09060',
```

```
31594 };
31595 var TOP_RANK_ROW_STYLE = {
31596   1: {
31597     fill: '#fff3cf',
31598     stroke: '#efbd48',
31599     medalCore: '#f8c84f',
31600     medalEdge: '#c88517',
31601     ribbon: '#e84848',
31602   },
31603   2: {
31604     fill: '#eaf2ff',
31605     stroke: '#b5cbe0',
31606     medalCore: '#d8e2ec',
31607     medalEdge: '#8aa3b9',
31608     ribbon: '#5f8fc1',
31609   },
31610   3: {
31611     fill: '#ffeadb',
31612     stroke: '#e1a078',
31613     medalCore: '#e79768',
31614     medalEdge: '#a15a2a',
31615     ribbon: '#c56a38',
31616   },
31617 };
31618 function _safeNum(v) {
31619   var n = parseInt(v, 10);
31620   return isNaN(n) ? 0 : n;
31621 }
31622 function _isCredentialError(err) {
31623   var msg = (err && (err.errMsg || err.message)) || '';
31624   return (err && err.err_code === 40001) || /invalid credential|access_token/i.test(msg);
31625 }
31626 function _parseWxgameData(kvList, tabKey) {
31627   if (!kvList || !kvList.length) return 0;
31628   var target = null;
31629   for (var i = 0; i < kvList.length; i++) {
31630     if (kvList[i] && kvList[i].key === tabKey) {
31631       target = kvList[i];
31632       break;
31633     }
31634   }
31635   if (!target) return 0;
31636   try {
31637     var obj = JSON.parse(target.value || '{}');
31638     if (obj && obj.wxgame && obj.wxgame.score != null) return _safeNum(obj.wxgame.score);
31639     if (obj && obj.score != null) return _safeNum(obj.score);
31640     return _safeNum(target.value);
31641   } catch (__) {
31642     return _safeNum(target.value);
31643   }
```



```
31644 }
31645 function _ensureAvatar(url) {
31646   if (!url) return null;
31647   var cached = state.avatarImgs[url];
31648   if (cached) return cached;
31649   try {
31650     var img = wx.createImage();
31651     img.onload = function () {
31652       state.avatarImgs[url] = img;
31653       _renderIfReady();
31654     };
31655     img.onerror = function () {
31656       state.avatarImgs[url] = null;
31657     };
31658     img.src = url;
31659     state.avatarImgs[url] = img;
31660     return img;
31661   } catch (_) {
31662     return null;
31663   }
31664 }
31665 function _ensureMedal(rank) {
31666   var src = MEDAL_ASSETS[rank];
31667   if (!src) return null;
31668   var cached = state.medallImgs[src];
31669   if (cached) return cached;
31670   try {
31671     var img = wx.createImage();
31672     img.onload = function () {
31673       state.medallImgs[src] = img;
31674       _renderIfReady();
31675     };
31676     img.onerror = function () {
31677       state.medallImgs[src] = null;
31678     };
31679     img.src = src;
31680     state.medallImgs[src] = img;
31681     return img;
31682   } catch (_) {
31683     return null;
31684   }
31685 }
31686 // ----- 数据拉取 -----
31687 /**
31688  * 拉好友 KV（不含定位自己用的 mine KV）。失败信息原样回传，让上层决定提示文案。
31689  * 拆出来是为了让 _getListForTab 能把它和 _fetchMineCloudStorage 并行起来。
31690  */
31691 function _fetchFriendCloudStorageOnly(tab, cb) {
31692   var meta = TAB_META[tab];
31693   if (!meta) {
```

```
31694     cb([], null);
31695     return;
31696 }
31697 if (typeof wx === 'undefined' || !wx.getFriendCloudStorage) {
31698     cb([], { kind: 'unsupported', msg: 'wx.getFriendCloudStorage missing' });
31699     return;
31700 }
31701 try {
31702     wx.getFriendCloudStorage({
31703         keyList: [meta.key],
31704         success: function (res) {
31705             var rows = (res && res.data) || [];
31706             var list = [];
31707             for (var i = 0; i < rows.length; i++) {
31708                 var r = rows[i];
31709                 var value = _parseWxgameData(r.KVDataList, meta.key);
31710                 if (value <= 0) continue;
31711                 list.push({
31712                     openid: r.openid || "",
31713                     nickname: r.nickname || '微信好友',
31714                     avatarUrl: r.avatarUrl || "",
31715                     value: value,
31716                 });
31717             }
31718             list.sort(function (a, b) { return b.value - a.value; });
31719             cb(list, null);
31720         },
31721         fail: function (err) {
31722             var errMsg = (err && (err.errMsg || err.message)) || "";
31723             var kind = 'empty';
31724             if (_isCredentialError(err)) kind = 'credential';
31725             else if (/privacy/i.test(errMsg)) kind = 'privacy';
31726             else if (/not support|unsupport/i.test(errMsg)) kind = 'unsupported';
31727             try { console.warn('[openData] getFriendCloudStorage fail', errMsg); } catch (_) {}
31728             cb([], { kind: kind, msg: errMsg });
31729         },
31730     });
31731 } catch (_) {
31732     cb([], null);
31733 }
31734 }
31735 /** 读取自己的微信 KV，用来在好友列表中定位"我"，并绘制底部固定个人信息行 */
31736 function _fetchMineCloudStorage(tab, cb) {
31737     var meta = TAB_META[tab];
31738     if (!meta || typeof wx === 'undefined' || !wx.getUserCloudStorage) {
31739         cb(0);
31740         return;
31741     }
31742     try {
31743         wx.getUserCloudStorage({
```

```
31744   keyList: [meta.key],
31745   success: function (res) {
31746     cb(_parseWxgameData((res && res.KVDataList) || [], meta.key));
31747   },
31748   fail: function (err) {
31749     if (!_isCredentialError(err)) {
31750       try { console.warn('[openData] getUserCloudStorage fail', err && (err.errMsg || err)); ...
31751     }
31752     cb(0);
31753   },
31754   });
31755 } catch (_) {
31756   cb(0);
31757 }
31758 }
31759 /**
31760 * 并行拉好友 KV + 自己 KV，并尽量"先来先用"地驱动渲染：
31761 * - 任一端先回 -> 立即触发一次 cb (partial=true, 主域已经能开始画骨架/局部内容)
31762 * - 两端都回 -> 再 cb 一次 (final, 写入缓存)
31763 *
31764 * 这样首屏体会比"两端都到齐再画"快几百毫秒，
31765 * 且兜底失败处理跟原先一致 (credential / privacy / unsupported 仍能透出) 。
31766 *
31767 * 通过 state.inflight[tab] 合并并发请求：prefetch + render 同时来时只发一组 wx 请求，
31768 * 多个 cb 都会在 partial / final 时被依次触发。
31769 */
31770 function _fetchFriendAndMineParallel(tab, cb) {
31771   if (!state.inflight) state.inflight = {};
31772   var running = state.inflight[tab];
31773   if (running) {
31774     running.cbs.push(cb);
31775     return;
31776   }
31777   state.loading = true;
31778   running = {
31779     friendDone: false,
31780     mineDone: false,
31781     friendList: [],
31782     friendErr: null,
31783     mineValue: 0,
31784     firedPartial: false,
31785     cbs: [cb],
31786   };
31787   state.inflight[tab] = running;
31788   function fireAll(list, err, mineV, isFinal) {
31789     var snapshot = running.cbs.slice();
31790     for (var i = 0; i < snapshot.length; i++) {
31791       try {
31792         snapshot[i](list, err, mineV, isFinal);
31793       } catch (e) {
```

```
31794     try { console.warn('[openData] cb throw', e); } catch (_) {}
31795   }
31796 }
31797 }
31798 function tryFire() {
31799   var allDone = running.friendDone && running.mineDone;
31800   if (!allDone && (running.friendDone || running.mineDone) && !running.firedPartial) {
31801     running.firedPartial = true;
31802     fireAll(running.friendList, running.friendErr, running.mineValue, /*final*/ false);
31803     return;
31804   }
31805   if (allDone) {
31806     state.loading = false;
31807     state.listCache[tab] = {
31808       ts: Date.now(),
31809       list: running.friendList,
31810       mineValue: running.mineValue,
31811       err: running.friendErr || undefined,
31812     };
31813     fireAll(running.friendList, running.friendErr, running.mineValue, /*final*/ true);
31814     delete state.inflight[tab];
31815   }
31816 }
31817 _fetchFriendCloudStorageOnly(tab, function (list, err) {
31818   running.friendDone = true;
31819   running.friendList = list || [];
31820   running.friendErr = err || null;
31821   tryFire();
31822 });
31823 _fetchMineCloudStorage(tab, function (v) {
31824   running.mineDone = true;
31825   running.mineValue = v || 0;
31826   tryFire();
31827 });
31828 }
31829 function _getListForTab(tab, forceRefresh, cb) {
31830   var cached = state.listCache[tab];
31831   if (!forceRefresh && cached && Date.now() - cached.ts < CACHE_TTL_MS) {
31832     cb(cached.list, cached.err, cached.mineValue || 0, /*final*/ true);
31833     return;
31834   }
31835   _fetchFriendAndMineParallel(tab, cb);
31836 }
31837 // ----- 通用绘图 -----
31838 function _clear() {
31839   if (!CTX || !CANVAS) return;
31840   CTX.clearRect(0, 0, CANVAS.width, CANVAS.height);
31841 }
31842 function _renderIfReady() {
31843   if (state.pendingRender) _render(state.pendingRender);
```

```
31844 }
31845 function _roundRect(x, y, w, h, r) {
31846   if (w < 2 * r) r = w / 2;
31847   if (h < 2 * r) r = h / 2;
31848   CTX.beginPath();
31849   CTX.moveTo(x + r, y);
31850   CTX.arcTo(x + w, y, x + w, y + h, r);
31851   CTX.arcTo(x + w, y + h, x, y + h, r);
31852   CTX.arcTo(x, y + h, x, y, r);
31853   CTX.arcTo(x, y, x + w, y, r);
31854   CTX.closePath();
31855 }
31856 /** 仿主域 PIXI.Text 的描边文字效果: 先描边再填充 */
31857 function _drawStrokedText(text, x, y, fontSize, weight, fill, stroke, strokeWidth) {
31858   CTX.font = weight + ' ' + fontSize + 'px ' + FONT_FALLBACK;
31859   if (strokeWidth > 0 && stroke) {
31860     CTX.lineWidth = strokeWidth;
31861     CTX.strokeStyle = stroke;
31862     CTX.lineJoin = 'round';
31863     CTX.strokeText(text, x, y);
31864   }
31865   CTX.fillStyle = fill;
31866   CTX.fillText(text, x, y);
31867 }
31868 function _drawHint(text, sub) {
31869   if (!CTX) return;
31870   var w = CANVAS.width;
31871   var h = CANVAS.height;
31872   var S = state.pixelRatio || 2;
31873   _clear();
31874   CTX.save();
31875   CTX.textAlign = 'center';
31876   CTX.textBaseline = 'middle';
31877   _drawStrokedText(
31878     text || '暂无好友数据',
31879     w / 2,
31880     h / 2 - 14 * S,
31881     26 * S,
31882     '900',
31883     COLOR.TEXT_DARK,
31884     COLOR.TEXT_LIGHT,
31885     3 * S
31886   );
31887   if (sub) {
31888     CTX.fillStyle = COLOR.HINT_SUB;
31889     CTX.font = '700 ' + (20 * S) + 'px ' + FONT_FALLBACK;
31890     CTX.fillText(sub, w / 2, h / 2 + 24 * S);
31891   }
31892   CTX.restore();
31893 }
```

```
31894 // ----- 主行渲染: 对齐主域 createRankRow / createRankBadge / createValueText -----
31895 /** 圆形头像: 白色底盘 + 描边, 未加载时只剩底盘 (自动 emoji 兜底放主域, 子域只画图) */
31896 function _drawAvatar(cx, cy, S, url, isSelf) {
31897     var r = 30 * S;
31898     // 白色底盘
31899     CTX.beginPath();
31900     CTX.arc(cx, cy, r, 0, Math.PI * 2);
31901     CTX.fillStyle = 'ffffff';
31902     CTX.fill();
31903     CTX.lineWidth = 3 * S;
31904     CTX.strokeStyle = isSelf ? COLOR.TEXT_ME_STROKE : COLOR.ROW_STROKE;
31905     CTX.stroke();
31906     var img = _ensureAvatar(url);
31907     if (img && img.width > 0) {
31908         CTX.save();
31909         CTX.beginPath();
31910         CTX.arc(cx, cy, r - 3 * S, 0, Math.PI * 2);
31911         CTX.clip();
31912         try {
31913             CTX.drawImage(img, cx - r, cy - r, r * 2, r * 2);
31914         } catch (_) {}
31915         CTX.restore();
31916     } else {
31917         // 加载未完成时给一个透明米色填充, 避免行间空白
31918         CTX.save();
31919         CTX.beginPath();
31920         CTX.arc(cx, cy, r - 3 * S, 0, Math.PI * 2);
31921         CTX.fillStyle = 'rgba(245,185,74,0.2)';
31922         CTX.fill();
31923         CTX.restore();
31924     }
31925 }
31926 /** 名次徽章: 前 3 名画"丝带 + 圆盘 + 数字"; 4+ 名显示纯数字 */
31927 function _drawBadge(cx, cy, S, rank, isSelf) {
31928     if (rank <= 3) {
31929         var palette = TOP_RANK_ROW_STYLE[rank];
31930         var medalImg = _ensureMedal(rank);
31931         if (medalImg && medalImg.width > 0) {
31932             try {
31933                 CTX.drawImage(medalImg, cx - 41 * S, cy - 41 * S, 82 * S, 82 * S);
31934                 return;
31935             } catch (_) {}
31936         }
31937         // 丝带 (两条三角形), 跟主域 createRankBadge 内坐标比例对齐
31938         CTX.fillStyle = palette.ribbon;
31939         CTX.beginPath();
31940         CTX.moveTo(cx - 14 * S, cy - 8 * S);
31941         CTX.lineTo(cx - 2 * S, cy - 8 * S);
31942         CTX.lineTo(cx - 2 * S, cy + 28 * S);
31943         CTX.lineTo(cx - 9 * S, cy + 22 * S);
```

```
31944 CTX.lineTo(cx - 16 * S, cy + 28 * S);
31945 CTX.closePath();
31946 CTX.fill();
31947 CTX.beginPath();
31948 CTX.moveTo(cx + 14 * S, cy - 8 * S);
31949 CTX.lineTo(cx + 2 * S, cy - 8 * S);
31950 CTX.lineTo(cx + 2 * S, cy + 28 * S);
31951 CTX.lineTo(cx + 9 * S, cy + 22 * S);
31952 CTX.lineTo(cx + 16 * S, cy + 28 * S);
31953 CTX.closePath();
31954 CTX.fill();
31955 // 圆盘：外圈 + 内圈
31956 CTX.beginPath();
31957 CTX.arc(cx, cy, 30 * S, 0, Math.PI * 2);
31958 CTX.fillStyle = palette.medalEdge;
31959 CTX.fill();
31960 CTX.beginPath();
31961 CTX.arc(cx, cy, 25 * S, 0, Math.PI * 2);
31962 CTX.fillStyle = palette.medalCore;
31963 CTX.fill();
31964 CTX.beginPath();
31965 CTX.arc(cx - 8 * S, cy - 9 * S, 8 * S, 0, Math.PI * 2);
31966 CTX.fillStyle = 'rgba(255,255,255,0.35)';
31967 CTX.fill();
31968 // 名次数字
31969 CTX.textAlign = 'center';
31970 CTX.textBaseline = 'middle';
31971 _drawStrokedText(
31972     String(rank),
31973     cx,
31974     cy,
31975     28 * S,
31976     '900',
31977     COLOR.TEXT_DARK,
31978     COLOR.TEXT_LIGHT,
31979     3 * S
31980 );
31981 return;
31982 }
31983 // 4+ 名：纯数字（自己行白字 + 深橙描边，其他玩家米色字 + 白描边）
31984 var display = isSelf && rank > 99 ? '99+' : String(rank);
31985 CTX.textAlign = 'center';
31986 CTX.textBaseline = 'middle';
31987 _drawStrokedText(
31988     display,
31989     cx,
31990     cy,
31991     (display === '99+' ? 26 : 32) * S,
31992     '900',
31993     COLOR.TEXT_RANK_NUM,
```

```
31994     COLOR.TEXT_LIGHT,
31995     2 * S
31996 );
31997 }
31998 /** 单行: 跟世界榜风格完全一致 */
31999 function _drawRow(rowX, rowY, rowW, rowH, S, meta, item, rank, isSelf) {
32000     var rowR = 16 * S;
32001     var topStyle = TOP_RANK_ROW_STYLE[rank];
32002     // 行底色 + 描边
32003     if (topStyle) {
32004         _roundRect(rowX, rowY, rowW, rowH, rowR);
32005         CTX.fillStyle = topStyle.fill;
32006         CTX.fill();
32007         CTX.lineWidth = 3 * S;
32008         CTX.strokeStyle = topStyle.stroke;
32009         CTX.stroke();
32010     } else if (isSelf) {
32011         _roundRect(rowX, rowY, rowW, rowH, rowR);
32012         CTX.fillStyle = '#ffedb0';
32013         CTX.fill();
32014         CTX.lineWidth = 3 * S;
32015         CTX.strokeStyle = '#efbd48';
32016         CTX.stroke();
32017     } else {
32018         CTX.beginPath();
32019         CTX.moveTo(rowX + 18 * S, rowY + rowH);
32020         CTX.lineTo(rowX + rowW - 18 * S, rowY + rowH);
32021         CTX.lineWidth = 1 * S;
32022         CTX.strokeStyle = 'rgba(233,216,185,0.75)';
32023         CTX.stroke();
32024     }
32025     // 名次徽章 / 圆形头像 / 名字 / 分数 的水平偏移:
32026     // 主域 createRankRow 内左基 = -w/2 + offset; 下面把它转成行左边距 + offset
32027     var badgeCx = rowX + 36 * S;
32028     var badgeCy = rowY + rowH / 2 + (topStyle ? -2 * S : 0);
32029     _drawBadge(badgeCx, badgeCy, S, rank, isSelf);
32030     var avatarCx = rowX + 112 * S;
32031     var avatarCy = rowY + rowH / 2;
32032     _drawAvatar(avatarCx, avatarCy, S, item.avatarUrl, isSelf);
32033     var nameX = rowX + 170 * S;
32034     var nameY = rowY + rowH / 2;
32035     CTX.textAlign = 'left';
32036     CTX.textBaseline = 'middle';
32037     var nick = (item.nickname || '微信好友').substring(0, 8);
32038     _drawStrokedText(
32039         nick,
32040         nameX,
32041         nameY,
32042         (topStyle ? 28 : 25) * S,
32043         '900',
```



```
32044     COLOR.TEXT_DARK,
32045     COLOR.TEXT_LIGHT,
32046     (topStyle ? 2 : 1) * S
32047 );
32048 var unit = meta.unit || "";
32049 var valText = item.value + unit;
32050 CTX.textAlign = 'right';
32051 CTX.textBaseline = 'middle';
32052 _drawStrokedText(
32053     valText,
32054     rowX + rowW - 32 * S,
32055     rowY + rowH / 2,
32056     26 * S,
32057     '900',
32058     '#d25a36',
32059     COLOR.TEXT_LIGHT,
32060     2 * S
32061 );
32062 }
32063 function _resolveMineRow(list, mineValue) {
32064     if (!list || !list.length) {
32065         return {
32066             item: { nickname: '自己', avatarUrl: "", value: mineValue || 0 },
32067             rank: mineValue > 0 ? 1 : 100,
32068         };
32069     }
32070     if (state.selfOpenId) {
32071         for (var i = 0; i < list.length; i++) {
32072             if (list[i].openid === state.selfOpenId) {
32073                 return { item: list[i], rank: i + 1 };
32074             }
32075         }
32076     }
32077     // 主域拿不到好友明文 openid 时，用自己的 KV 分数回定位。
32078     // 极少数同分会命中第一个同分好友，但底部仍能保持世界榜同款“个人信息”布局。
32079     if (mineValue > 0) {
32080         for (var j = 0; j < list.length; j++) {
32081             if (list[j].value === mineValue) {
32082                 return { item: list[j], rank: j + 1 };
32083             }
32084         }
32085         var rank = 1;
32086         for (var k = 0; k < list.length; k++) {
32087             if (list[k].value > mineValue) rank += 1;
32088         }
32089         return {
32090             item: { nickname: '自己', avatarUrl: "", value: mineValue },
32091             rank: rank,
32092         };
32093     }
```

```
32094 return {
32095   item: { nickname: '自己', avatarUrl: '', value: 0 },
32096   rank: 100,
32097 };
32098 }
32099 function _drawBoard(list, mineValue) {
32100   var S = state.pixelRatio || 2;
32101   var w = CANVAS.width;
32102   var h = CANVAS.height;
32103   // 与主域 drawList + drawMineRow 完全一致：上方列表预留底部个人信息行。
32104   var rowH = 84 * S;
32105   var gap = 10 * S;
32106   var startY = 0;
32107   var mineGap = 26 * S;
32108   var listH = Math.max(0, h - rowH - mineGap);
32109   var rawScrollY = state.scrollY || 0;
32110   var totalH = list.length * (rowH + gap);
32111   var minScrollY = Math.min(0, listH - startY - totalH);
32112   var scrollY = Math.max(minScrollY, Math.min(0, rawScrollY));
32113   var meta = TAB_META[state.tab] || TAB_META.bowl;
32114   var mine = _resolveMineRow(list, mineValue);
32115   _clear();
32116   CTX.save();
32117   CTX.beginPath();
32118   CTX.rect(0, 0, w, listH);
32119   CTX.clip();
32120   for (var i = 0; i < list.length; i++) {
32121     var item = list[i];
32122     var ry = startY + i * (rowH + gap) + scrollY;
32123     if (ry + rowH < 0 || ry > listH) continue;
32124     var isSelf = (state.selfOpenId && item.openid === state.selfOpenId)
32125       || (mineValue > 0 && item.value === mineValue && i + 1 === mine.rank);
32126     _drawRow(0, ry, w, rowH, S, meta, item, i + 1, isSelf);
32127   }
32128   CTX.restore();
32129   // 底部固定个人信息行：位置、尺寸、橙色高亮都按世界榜 drawMineRow 对齐。
32130   _drawRow(0, h - rowH, w, rowH, S, meta, mine.item, mine.rank, true);
32131 }
32132 /**
32133  * 把"拿到的数据状态"翻成一次具体的绘制。
32134  * 拆开是因为并行拉取时会被回调两次（partial + final），两次都要走同一套画法。
32135  */
32136 function _paintResult(list, err, mineValue) {
32137   if (err && err.kind === 'privacy') {
32138     _drawHint('好友榜暂不可用', '请检查小程序后台隐私协议配置');
32139     return;
32140   }
32141   if (err && err.kind === 'credential') {
32142     _drawHint('好友榜暂不可用', '微信凭证异常，重进小程序后再试');
32143     return;
```

```
32144 }
32145 if (err && err.kind === 'unsupported') {
32146   _drawHint('好友榜暂不可用, '请升级微信到最新版本重试');
32147   return;
32148 }
32149 if (!list || !list.length) {
32150   if (mineValue > 0) {
32151     _drawBoard([], mineValue);
32152   } else {
32153     _drawHint('暂无好友上榜, '邀请微信好友一起来玩即可上榜');
32154   }
32155   return;
32156 }
32157 _drawBoard(list, mineValue || 0);
32158 }
32159 function _render(msg) {
32160   state.pendingRender = msg;
32161   if (!CTX || !CANVAS) {
32162     try { console.warn('[openData] render skipped: no CTX/CANVAS'); } catch (_) {}
32163     return;
32164   }
32165   state.tab = msg.tab || state.tab;
32166   state.pixelRatio = msg.pixelRatio || state.pixelRatio;
32167   state.scrollY = msg.scrollY !== null ? msg.scrollY : state.scrollY;
32168   state.selfOpenId = msg.selfOpenId || state.selfOpenId;
32169   // 新基础库下 sharedCanvas 的 width/height 在 openDataContext 内只读,
32170   // 必须由主域写入 (见 src/utls/friendRanking.ts:ensureSharedCanvasSize) 。
32171   // 进入这一帧能拿到啥就先画啥, 绝不留白屏:
32172   // - 有缓存 -> 直接用缓存数据画一份 (不管 force, 避免 force 重拉时白屏空等) ;
32173   // - 无缓存 -> 画一个 loading 骨架占位。
32174   // 后续 _getListForTab 拉到新数据时会再覆盖一次 (partial / final 两次都会刷) 。
32175   var cached = state.listCache[state.tab];
32176   var hasFresh = cached && Date.now() - cached.ts < CACHE_TTL_MS;
32177   try {
32178     console.log(
32179       '[openData] render tab=' + state.tab + ' force=' + !!msg.force
32180       + ' hasFreshCache=' + !!hasFresh
32181       + ' canvas=' + (CANVAS.width + 'x' + CANVAS.height)
32182     );
32183   } catch (_) {}
32184   if (hasFresh) {
32185     _paintResult(cached.list, cached.err, cached.mineValue || 0);
32186   } else if (!msg.silent) {
32187     _drawHint('正在加载好友榜...', '正在向微信请求好友数据');
32188   }
32189   _getListForTab(state.tab, !!msg.force, function (list, err, mineValue, isFinal) {
32190     if (!isFinal && (!list || !list.length) && !err && !(mineValue > 0)) {
32191       return;
32192     }
32193     try {
```

```
32194 console.log(
32195   '[openData] paint tab=' + state.tab
32196   + ' isFinal=' + isFinal
32197   + ' listLen=' + ((list && list.length) || 0)
32198   + ' mineValue=' + (mineValue || 0)
32199   + ' err=' + (err ? err.kind : 'none')
32200 );
32201 } catch (_) {}
32202 _paintResult(list, err, mineValue);
32203 });
32204 }
32205 /**
32206  * 静默预拉：把当前 tab 的好友/自己 KV 拉到子域 60s 缓存里，但不画任何东西。
32207  *
32208  * 主域 LeaderboardScene 在还停留在世界榜时调用，等用户切到好友榜时直接命中缓存，
32209  * 跳过整段 wx 网络往返。如果用户从不切到好友榜，多出来的两次 KV 网络也只是一次性的，
32210  * 不会持续刷。
32211  */
32212 function _prefetch(tab) {
32213   if (!tab) tab = state.tab;
32214   state.tab = tab || state.tab;
32215   var cached = state.listCache[state.tab];
32216   if (cached && Date.now() - cached.ts < CACHE_TTL_MS) return;
32217   _fetchFriendAndMineParallel(state.tab, function () {
32218     // 结果已经写进 state.listCache 由 _fetchFriendAndMineParallel 负责，无需绘制
32219   });
32220 }
32221 // ----- 消息入口 -----
32222 try {
32223   console.log(
32224     '[openData] subdomain boot, hasCANVAS=' + !!CANVAS + ' hasCTX=' + !!CTX
32225     + ' wx=' + (typeof wx !== 'undefined')
32226     + ' wxOnMessage=' + (typeof wx !== 'undefined' && typeof wx.onMessage === 'function')
32227   );
32228 } catch (_) {}
32229 if (typeof wx !== 'undefined' && wx.onMessage) {
32230   wx.onMessage(function (data) {
32231     try { console.log('[openData] onMessage action=' + (data && data.action)); } catch (_) {}
32232     if (!data || !data.action) return;
32233     if (data.action === 'render' || data.action === 'refresh') {
32234       _render(data);
32235     } else if (data.action === 'prefetch') {
32236       _prefetch(data.tab);
32237     } else if (data.action === 'invalidate') {
32238       // 主域刚 setUserCloudStorage 完，下次 render 时强制重新拉。
32239       // 在飞请求已经发出了，让它完成并写入新缓存即可；这里只清旧缓存。
32240       state.listCache = {};
32241     }
32242   });
32243 }
```